CS702PC: COMPILER DESIGN B.Tech. IV Year I Sem.

UNIT - I Introduction: The structure of a compiler, the science of building a compiler, programming language basics **Lexical Analysis**: The Role of the Lexical Analyzer, Input Buffering, Recognition of Tokens, The Lexical-Analyzer Generator Lex, Finite Automata, From Regular Expressions to Automata, Design of a Lexical-Analyzer Generator, Optimization of DFA-Based Pattern Matchers.

UNIT - II Syntax Analysis: Introduction, Context-Free Grammars, Writing a Grammar, Top-Down Parsing, Bottom-Up Parsing, Introduction to LR Parsing: Simple LR, More Powerful LR Parsers, Using Ambiguous Grammars and Parser Generators.

UNIT - III Syntax-Directed Translation: Syntax-Directed Definitions, Evaluation Orders for SDD's, Applications of Syntax-Directed Translation, Syntax-Directed Translation Schemes, Implementing L-Attributed SDD's. **Intermediate-Code Generation**: Variants of Syntax Trees, Three-Address Code, Types and Declarations, Type Checking, Control Flow, Switch-Statements, Intermediate Code for Procedures.

UNIT - IV Run-Time Environments: Stack Allocation of Space, Access to Nonlocal Data on the Stack, Heap Management, Introduction to Garbage Collection, Introduction to Trace-Based Collection. **Code Generation**: Issues in the Design of a Code Generator, The Target Language, Addresses in the Target Code, Basic Blocks and Flow Graphs, Optimization of Basic Blocks, A Simple Code Generator, Peephole Optimization, Register Allocation and Assignment, Dynamic Programming Code-Generation

UNIT - V **Machine-Independent Optimization**: The Principal Sources of Optimization, Introduction to Data-Flow Analysis, Foundations of Data-Flow Analysis, Constant Propagation, Partial-Redundancy Elimination, Loops in Flow Graphs.

List of Experiments

1. Implementation of symbol table.
2. Develop a lexical analyzer to recognize a few patterns inc (ex. Identifiers, constants, comments, operators etc.)
3. Implementation of lexical analyzer using lex tool.
4. Generate yacc specification for a few syntactic categories.

a) Program to recognize a valid arithmetic expression that uses operator +,-, * and /.

b) Program to recognize a valid variable which starts with a letter followed by any number of letter or digits.

c) Implementation of calculator using lex and yacc.

5. Convert the bnf rules into yacc form and write code to generate abstract syntax tree.
6. Implement type checking
7. Implement any one storage allocation strategies (heap, stack, static)
8. Write a lex program to count the number of words and number of lines in a given file or program.
9. Write a 'C' program to implement lexical analyzer using c program.
10. write recursive descent parser for the grammar E->E+T E->T T->T*F T->F F->(E)/id.
11. write recursive descent parser for the grammar S->(L) S->a L->L,S L->S
12. Write a C program to calculate first function for the grammar E->E+T E->T T->T*F T->F F->(E)/id
13. Write a YACC program to implement a top down parser for the given grammar.
13. Write a YACC program to evaluate algebraic expression.

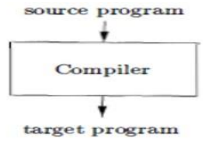
UNIT – 1

➢ LANGUAGE PROCESSORS

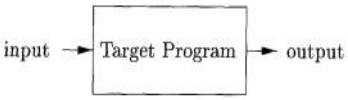
Language processors are software tools that translate high-level programming languages into machine code that computers can understand and execute. They bridge the gap between human-readable code and the instructions a computer can directly process. Common types of language processors include compilers, interpreters, and assemblers.

a) Compilers:

Compilers translate the entire source code of a program into machine code (target program) at once before execution. They check for errors in the entire code and report them at the end. If there are no errors, the compiled code can be executed directly. Examples include C compilers.

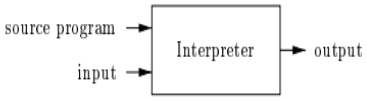


If the target program is an executable machine-language program, it can then be called by the user to process inputs and produce outputs.



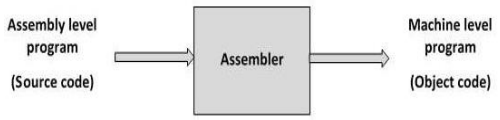
b) Interpreters:

Interpreters translate and execute code line by line. They translate a single line, execute it, and then move on to the next. If an error is encountered during execution, the process stops. Interpreters can be useful for debugging and testing code quickly. Examples include Python interpreters.



c) Assemblers:

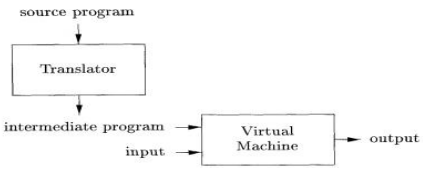
Assemblers translate assembly language (a low-level programming language) into machine code. Assembly language is closer to machine code than high-level languages like C++ or Python.



d) Hybrid

A hybrid compiler combines aspects of both compilation and interpretation.

For example, a java language processor uses both compilation and interpretation. First the source code is translated into intermediate code and then the output along with input is used to generate output. This kind of compiler is called as hybrid compiler.



➢ LANGUAGE PROCESSING SYSTEM

A language processing system in compiler design refers to the *sequence of software tools* and *components* that work together to transform a source program written in a high-level language into an executable program that a computer can understand and run. This system typically involves several stages:

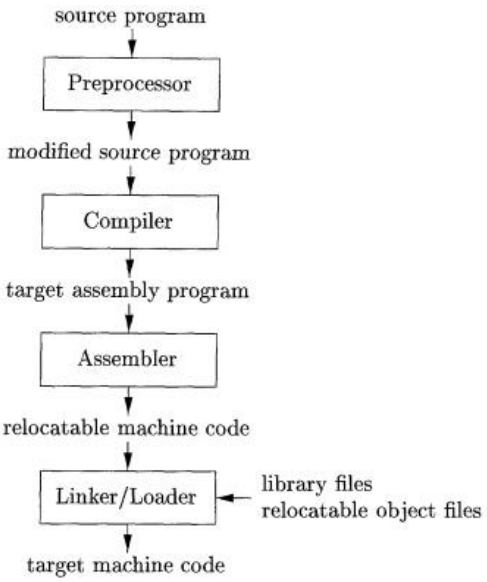
Preprocessor

This component handles pre-processor directives like *macro expansion, file inclusion, and conditional compilation*. It produces a modified source program/pure High level Language that is then fed to the compiler.

Compiler

The modified source program is then fed to a compiler. The compiler may produce an assembly-language program as its outpu t, because assembly language is easier to produce as output and is easier to debug.

The assembly language is then processed by a program called an *assembler* that produces relocatable machine code as its output.



Assembler

The output of an assembler is called an object file, which contains a combination of machine instructions as well as the data required to place these instructions in memory(relocatable).

Linker

Linker is a computer program that *links and merges* various object files together in order to make an executable file. All these files might have been compiled by separate assemblers. The major task of a linker is to search and locate referenced module/routines in a program and to determine the memory location where these codes will be loaded, making the program instruction to have absolute references.

Loader

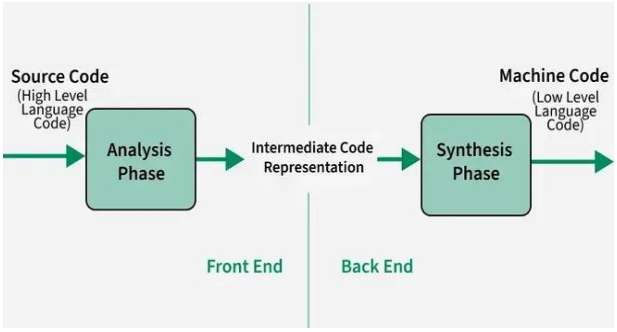
Loader is a part of operating system and is responsible for loading executable files into memory and execute them. It calculates the size of a program (instructions and data) and creates memory space(absolute) for it. It initializes various registers to initiate execution.

➢ THE STRUCTURE / PHASES OF COMPILER

The compilation process is a sequence of various phases. Each phase takes input from its previous stage, has its own representation of source program, and feeds its output to the next phase of the compiler.

The 6 phases of a compiler are:

1. Lexical Analysis
2. Syntactic Analysis or Parsing
3. Semantic Analysis
4. Intermediate Code Generation
5. Code Optimization
6. Code Generation



1 Lexical Analysis

The first phase of scanner works as a text scanner. This phase scans the source code as a stream of characters and converts it into meaningful lexemes. Lexical analyzer represents these lexemes in the form of tokens. For example lexical errors are spelling mistakes., exceeding length of identifier, illegal characters usage.

2. Syntax Analysis

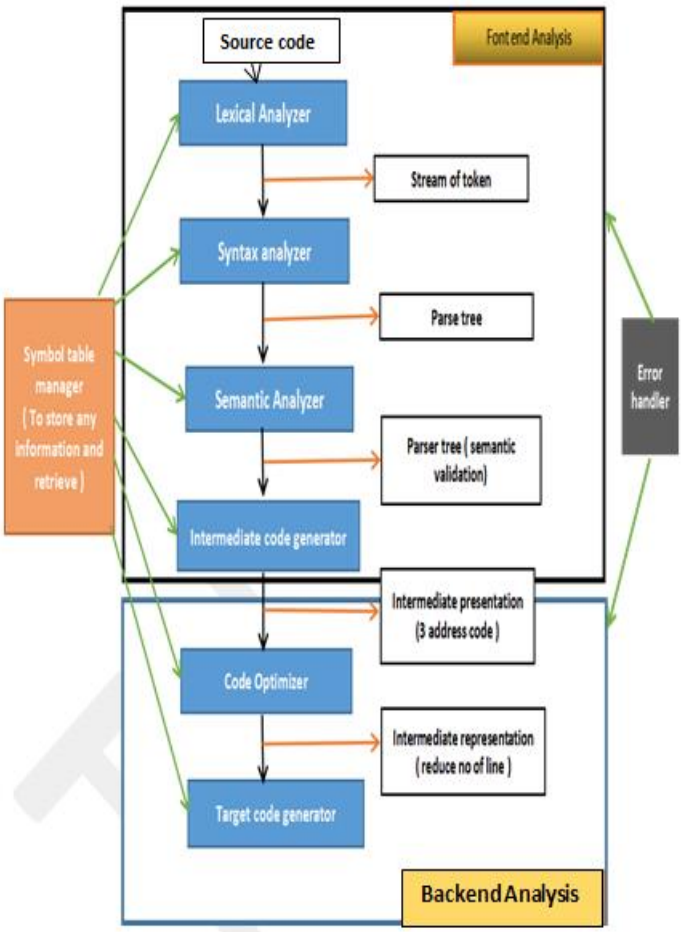
The next phase is called the syntax analysis or **parsing**.

It takes the token produced by lexical analysis as input and generates a parse tree (or syntax tree). In this phase, *token arrangements are checked* against the *source code grammar.* Checks whether the program follows the grammatical rules of the language.

For example, Missing semicolons, unmatched parentheses, incorrect statement structure

3. Semantic Analysis

Semantic analysis checks whether the statements have meaningful and valid usage. For example, assignment of values is between compatible data types, and adding string to an integer. Also, the semantic analyzer keeps track of identifiers, their types and expressions; whether identifiers are declared before use or not etc. The semantic analyzer produces an annotated syntax tree as an output. Saves collected data to symbol tables or syntax trees. For example, Type mismatch, undeclared variables, multiple declarations, incorrect function arguments.



4. Intermediate Code Generation

After semantic analysis the compiler generates an intermediate code of the source code for the target machine. It represents a program for some abstract machine. It is in between the high-level language and the machine language. This intermediate code should be generated in such a way that it makes it easier to be translated into the target machine code. An intermediate code generator develops three address codes. This stage serves as a bridge or way from analysis to synthesis.

5. Code Optimization

The next phase does code optimization of the intermediate code. Optimization can be assumed as something that removes unnecessary code lines, and arranges the sequence of statements in order to speed up the program execution without wasting resources (CPU, memory).

6. Code Generation

In this phase, the code generator takes the optimized representation of the intermediate code and maps it to the target machine language. The code generator translates the intermediate code into a sequence of (generally) re-locatable machine code. Sequence of instructions of machine code performs the task as the intermediate code would do.

Symbol Table

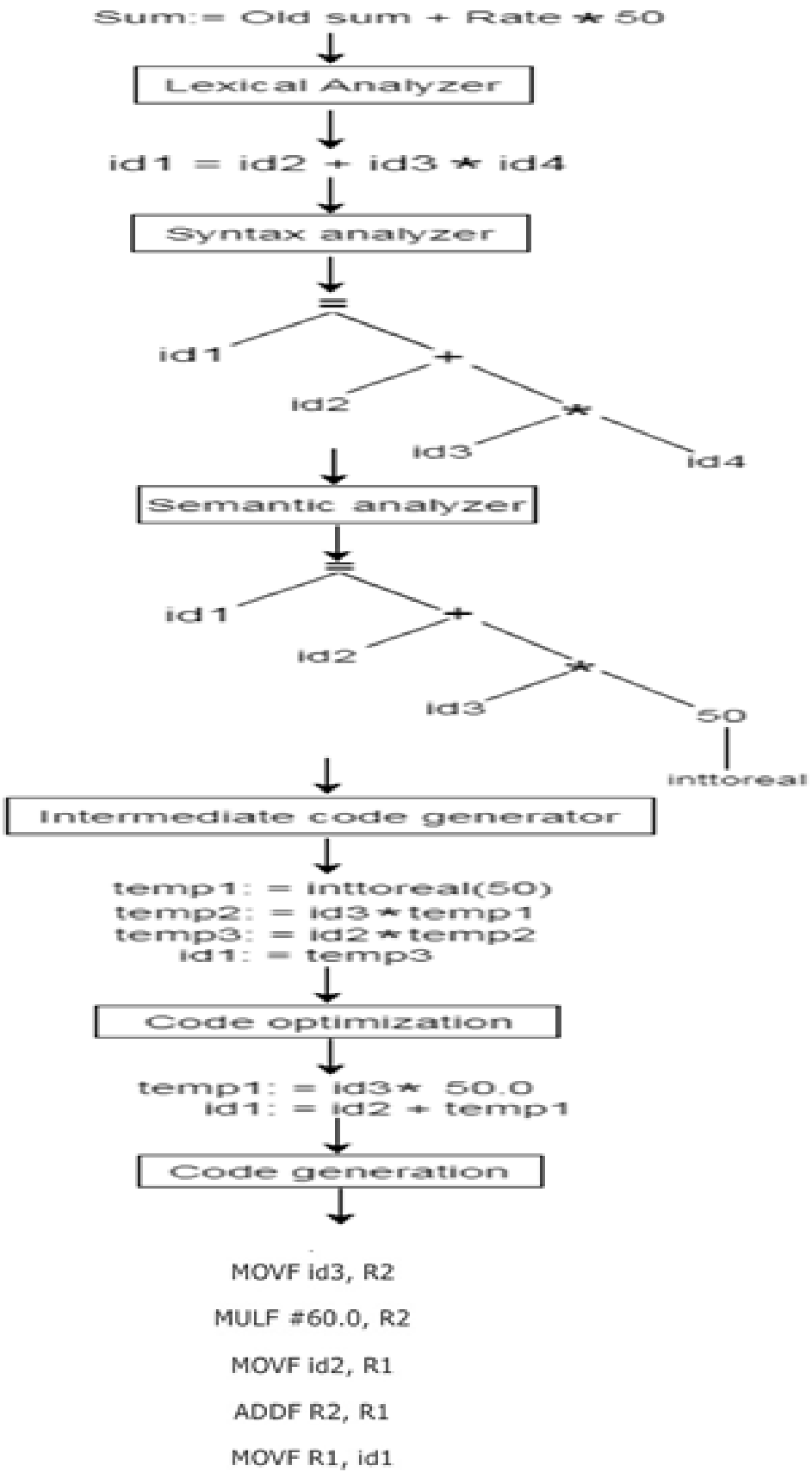
It is a data-structure maintained throughout all the phases of a compiler. All the identifier's names along with their types are stored here. The symbol table makes it easier for the compiler to quickly search the identifier record and retrieve it. The symbol table is also used for scope management.

The symbol table connects or interacts with all phases of the compiler and error handler for updates.

Error Handling and Reporting:

Each phase can encounter errors. After detecting an error, a phase must deal that error, so that compilation can proceed, allowing further errors to be detected.

The output of different phases of compiler for a statement Sum = OldSum +Rate*60



➢ TOKENS ,PATTERNS, LEXEMES:

• Token is a sequence of characters in the input that form a meaningful word. In most languages, the tokens fall into these categories: Keywords , Operators, Identifiers , Constants, Literal strings and Punctuation.

Non-tokens are comments, tabs, blanks, newline

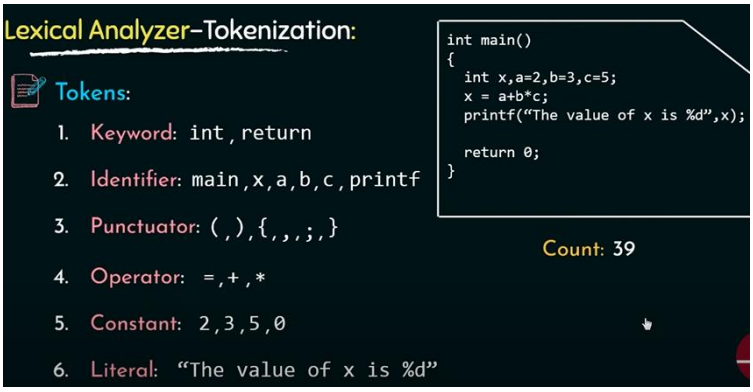
• There is a set of strings in the input for which a token is produced as output. This set is described a rule called *pattern*. i.e. rule describing token

• A *lexeme* is a sequence of characters in the source program that is matched by the pattern for a token

Token = name of a category.

Pattern = rule used to recognize that category.

Lexeme = actual characters in the source program that belong to that category.



➢ THE SCIENCE OF BUILDING A COMPILER

A compiler must accept all source programs that conform to the specification of the language; the set of source programs is infinite and any program can be very large, consisting of possibly millions of lines of code. Any transformation performed by the compiler while translating a source program must preserve the meaning of the program being compiled.

Modelling in compiler design and implementation:

The study of compilers is mainly a study of how we design the right mathematical models and choose the right algorithms. Some of most fundamental models are finite-state machines and regular expressions. These models are useful for describing the lexical units of programs (keywords, identifiers, and such) and for describing the algorithms used by the compiler to recognize those units. Also among the most fundamental models are context-free grammars, used to describe the syntactic structure of programming languages such as the nesting of parentheses or control constructs. Similarly, trees are an important model for representing the structure of programs and their translation into object code.

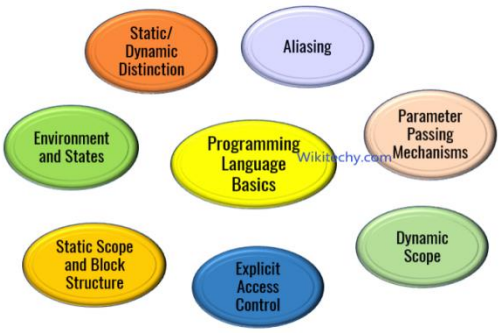
The science of code optimization:

The term "optimization" in compiler design refers to the attempts that a compiler makes to produce code that is more efficient than the obvious code.. It is more complex because processor architectures have become more complex, yielding more opportunities to improve the way code executes. Compiler optimizations must meet the following design objectives: 1. The optimization must be correct, that is, preserve the meaning of the compiled program, 2. The optimization must improve the performance of many programs, 3. The compilation time must be kept reasonable, and 4. The engineering effort required must be manageable.

Thus, in studying compilers, we learn not only how to build a compiler, but also the general methodology of solving complex and open-ended problems.

➢ PROGRAMMING LANGUAGE BASICS

Programming language basics are fundamental to compiler design because the primary function of a compiler is to translate programs written in a high-level programming language into machine code or another lower-level representation. To perform this translation correctly and efficiently, the compiler must understand the structure, semantics, scope rules, and execution behaviour of the source language.



Programming Language Basics

1. Static and Dynamic Distinction

Programming languages use either static policies or dynamic policies to make decisions regarding variables, types, binding, and memory allocation.

Static Policy

If a language allows the compiler to determine an issue before execution, it is said to use a static policy. Such decisions are made at compile time.

Examples:

Static scope (Lexical scope), Static type checking (C, C++, Java), Static binding, Static memory allocation

Dynamic Policy

If a decision can be made only during program execution, the language uses a dynamic policy. Such decisions are made at run time.

Examples:

Dynamic scope, Dynamic type checking, Dynamic binding, Dynamic memory allocation

2. Scope of a Declaration

The scope of a declaration of a variable is the region of a program in which references to that variable are valid. It determines where a variable can be accessed

3. Static Scope (Lexical Scope)

A language uses static scope or lexical scope if the scope of a variable can be determined by examining the program text alone.

In static scoping, when a variable is referenced, the compiler searches: The current block, The enclosing blocks, The global scope.

The variable declaration in the nearest enclosing block is selected.

Languages Using Static Scope – C, C++, Java, C#

4. Dynamic Scope

In dynamic scoping, the scope of a variable is determined during program execution based on the sequence of function calls. A reference to a variable refers to the declaration in the most recently called procedure containing that declaration.

Scope depends on the calling sequence. Same variable reference may refer to different declarations at different times.

Applications - Macro expansion in the C preprocessor. Method resolution in object-oriented programm

5. Environment and State

In compiler design, environment and state are important concepts for managing variables and their values. An environment is a mapping from variable names to memory locations.

It answers the question: "Where is a variable stored?"

State

A state is a mapping from memory locations to their current values. It answers the question: "What value is stored in a location?". As the program executes, the state changes whenever variable values are updated.

6. Static Scope and Block Structure

A block is a group of declarations and statements treated as a single unit. In C and C++, blocks are enclosed by braces { }. The scope rules in C are based on the program structure. Therefore, the scope of a variable is determined by where it is declared.

Programming languages such as C++, Java, and C# also provide explicit control over scopes through access specifiers.

7. Explicit Access Control

Object-oriented languages provide explicit access control to restrict the visibility of class members. This supports encapsulation.

Public : Accessible from anywhere. Visible inside and outside the class.

Private : Accessible only within the same class. Hidden from outside classes.

Protected : Accessible within the class and its subclasses.

8. Parameter Passing Mechanisms

Parameter passing mechanisms define how actual parameters are associated with formal parameters during a procedure call.

Formal Parameter : The variable declared in the function definition.

Actual Parameter : The value or variable supplied during the function call.

(a) Call by Value :

In call by value, a copy of the actual parameter is passed to the function.

Changes made inside the function do not affect the original variable.

(b) Call by Reference

In call by reference, the address of the actual parameter is passed. Changes made inside the function affect the original variable.

(c) Call by Name

In call by name, the actual parameter behaves as if it were substituted directly for the formal parameter. This mechanism is mainly used in some older languages.

9. Aliasing

Aliasing occurs when two or more variables refer to the same memory location. This commonly occurs in call-by-reference parameter passing. If both parameters refer to the same variable, they become aliases of one another.

➢ THE ROLE OF THE LEXICAL ANALYZER

The **lexical analysis** is the first phase of the compiler where a lexical analyser operate as an interface between the source code and the rest of the phases of a compiler.

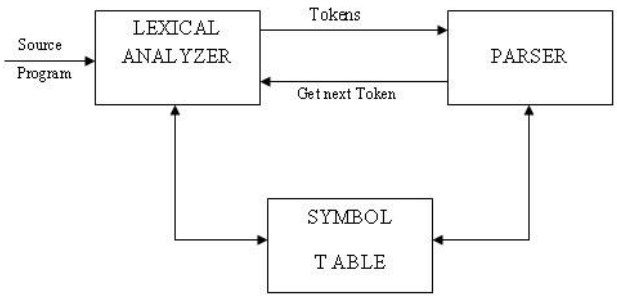
A Lexical Analyzer, also known as a *scanner*, is responsible for reading the source code character by character and converting it into meaningful tokens. These tokens are then used by the next stage of the compiler for further processing.

For example, if we have the following line of code in C:

int num = 10;

The **Lexical Analyzer** breaks this into tokens such as:

Lexeme	Token
Int	Keyword
num	Identifier
=	Operator
10	Numeric constant
;	Special symbol



Source program → Lexical analtzer → Lexemes → Tokens

Role of Lexical Analyzer

The lexical analyzer performs the following tasks-

- The lexical analyzer is responsible for removing the white spaces and comments from the source program.
- The lexical analyzer identifies lexical errors, such as invalid characters or malformed tokens, and reports them to the user,
- Interaction with the Parser: It provides a stream of tokens to the next phase, the parser (syntax analyzer), which then uses these tokens to build a parse tree
- Tokenizer : The lexical analyzer reads the source code character by character and groups these characters into meaningful units called "tokens.".
- Symbol Table Management:

➢ ATTRIBUTES FOR TOKENS

In compiler design, a token is the smallest meaningful unit of a program, such as an identifier, keyword, constant, or operator. Each token has an attribute, which contains extra information about that token.

Token attributes are additional details associated with a token that help the compiler to understand and process the token correctly during different phases of compilation. These attributes

1. Provide detailed information about the tokens to the parser.
2. Store semantic values that are needed to evaluate expressions or generate code.
3. Differentiate between tokens of the same type but with unique values.

The lexical analyzer typically stores token attributes in a symbol table or passes them to the next stage of the compiler as part of the token data.

For example, if the input code is: *int x = 10;*

The lexical analyzer might produce tokens and their attributes like this:

Lexeme	*Token Type*	*Attribute*
int	Keyword	None
x	Identifier	Name: x
=	Operator	Type: assignment
10	Literal	Value: 10
;	Symbol	None

Lexical errors

A lexical error in compiler design occurs when the lexer (also known as the lexical analyzer) encounters a sequence of characters in the source code that it cannot recognize as a valid token. Essentially, it's a failure to match a string of characters against any predefined token patterns.
Types of lexical error that can occur in a lexical analyzer are as follows:

1. Exceeding length of identifier or numeric constants.
2. Appearance of illegal characters
3. Unmatched string
4. Spelling Error
5. Removal of the character that should be present.

➢ LEXICAL ANALYSIS VS PARSING

Lexical Analysis

- In compiler design, lexical analysis and parsing are distinct phases that work together to translate source code into a form that a computer can understand.
- It is First phase of compilation process
- Lexical analysis, also known as scanning, converts the source code into a stream of tokens. It identifies meaningful tokens without regard to how they relate to one another.
- lexical analysis reads the source code one character at a time and converts it into meaningful lexemes (tokens)
- *Input Buffering* is used for scanning the source Code
- Regular expressions and finite automata are used in this phase.
- LEX is an automated tool which is used to generate lexical analyzer

Parsing

- Parsing, or syntax analysis, takes these tokens and checks if they are arranged according to the language's grammar, building a tree-like structure (parse tree or abstract syntax tree) that represents the code's structure.
- It is Second phase of the compilation process
- When an error is found, a parser must produce a diagnostic message and recover. Recovery is required so that the compiler finds as many errors as possible.
- Context Free grammar is used in this phase
- Top-down and Bottpm-up approaches are used for parse tree construction
    - Top-down parsers build the parse tree from the root downwards to the leaves.
    - Bottom-up parsers build the parse tree from the leaves upwards to the root
- YACC is an automated tool which is used to generate syntax analyser

➢ INPUT BUFFERING

A buffer is a region of memory used to temporarily store the source code.

Input buffering in compiler design is a technique to improve the efficiency of reading source code by storing it in memory buffers

Reading source code character by character can be slow. Input buffering allows *the compiler to read larger chunks of code into memory (buffers) and then process them*, reducing the number of disk or file access operations.

Advantages

Improves Efficiency

Reduced I/O overhead

Simplified lexical analysis

There are two methods used in this context: One Buffer Scheme, and Two Buffer Scheme.

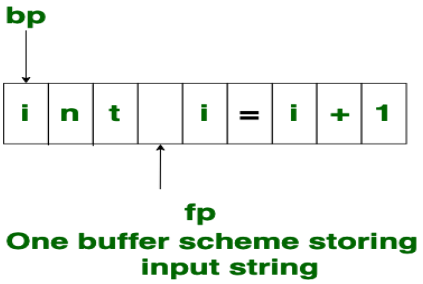
i) One Buffer Scheme:

- **Pointers:**

The lexical analyzer scans the input from left to right one character at a time. It uses two pointers begin ptr(**bp**) and forward ptr(**fp**) to keep track of the pointer of the input scanned.

Two pointers are typically used within the buffer:

- Initially both the pointers point to the first character of the input string. The forward ptr moves ahead to search for end of lexeme. As soon as the blank space is encountered, it indicates end of lexeme.
- Once a valid lexeme is identified (e.g., a keyword, identifier, operator), the characters between lexemeBegin and the position of the forward pointer (or just before it, depending on the implementation) constitute the recognized lexeme.



One buffer scheme storing input string

- After a lexeme is recognized and processed (e.g., converted into a token and passed to the parser), the lexemeBegin pointer is advanced to the character immediately following the newly found lexeme. The forward pointer is also reset to this new lexemeBegin position, ready to begin scanning for the next lexeme.

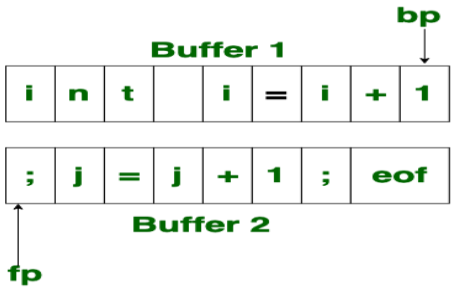
In this scheme, only one buffer is used to store the input string but the problem with this scheme is that if lexeme is very long then it crosses the buffer boundary, to scan rest of the lexeme the buffer has to be refilled, that makes overwriting the first of lexeme.

ii) Two-buffer scheme:

A common approach is to use two buffers. While one buffer is being processed by the lexical analyzer, the other can be filled with the next chunk of code.

It also known as Buffer Pairs.

The scheme utilizes two buffers, typically of equal size, to store portions of the source code. Let's call them Buffer 1 and Buffer 2.



Two buffer scheme storing input string

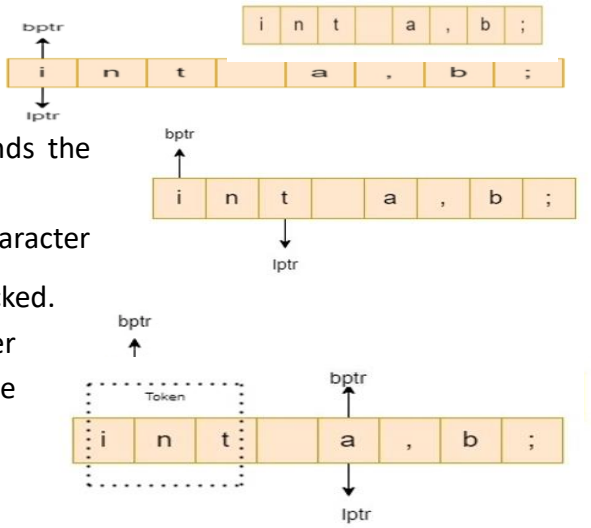
- Initially, Buffer 1 is filled with a block of characters from the source file.
- The lexical analyzer starts processing characters from Buffer 1 using the forward pointer.

- When the forward pointer reaches the end of Buffer 1 (marked by a sentinel character like EOF), the system switches to Buffer 2.
- Simultaneously, while Buffer 2 is being processed, Buffer 1 is refilled with the next block of characters from the source file.
- This process alternates between Buffer 1 and Buffer 2, ensuring a continuous flow of input.

The standard approach uses two buffers of equal size (often 4096 bytes each), with two pointers:

Sample Example

For the statement **int a,b;**



Both points begin at the start of the string that is saved in the buffer.

The forward Pointer examines the buffer until it finds the token.

Before the token ("int") can be identified, the character ("blank space") beyond the token ("int") must be checked.

Both pointers will be set to the next token ('a') after processing the token ("int"), and this procedure will be continued throughout the program.

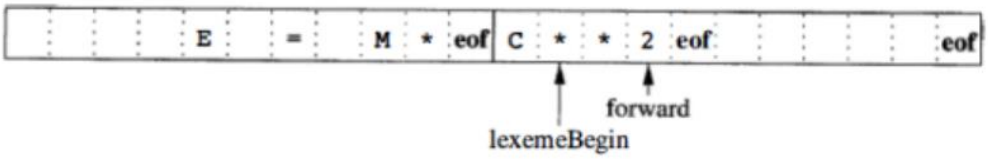
Sentinels:

A sentinel is a special end-marker placed at the end of each input buffer in lexical analysis to indicate the buffer boundary and eliminate the need for checking the end of the buffer after every character, thereby improving scanning efficiency.

For each character read, we make two tests: one for the end of the buffer, and one to determine what character is read. We can combine the buffer-end test with the test for the current character if we extend each buffer to hold a sentinel character at the end.

The sentinel is a special character that cannot be part of the source program, and a natural choice is the character eof.

The sentinel arrangement is as shown below:



➢ SPECIFICATION OF TOKEN

In compiler design, there are three specifications of token-

1. String 2. Language 3. Regular Expressions

1. Strings

Strings are a finite set of symbols or characters. These symbols can be a digit or an alphabet. There is also an empty string which is denoted by **ε**.

Operations on String

The operations that can be performed on a string are-

a) A **prefix** of string s is any string obtained by removing zero or more symbols from the end of string s. For example, *ban* is a prefix of *banana*.

b) A **suffix** of string s is any string obtained by removing zero or more symbols from the beginning of s. For example, *nana* is a suffix of *banana*.
c) A **substring** of s is obtained by deleting any prefix and any suffix from s. For example, *nan* is a substring of *banana*.
d) The **proper prefixes, suffixes, and substrings** of a string s are those prefixes, suffixes, and substrings, respectively of s that are not ε or not equal to s itself. For example, the proper prefixes of the word "happy" are "h", "ha", and "hap", but not "happy".
e) A **subsequence** of s is any string formed by deleting zero or more not necessarily consecutive positions of s. For example, baan is a subsequence of banana.
f) **Concatenation** is defined as the addition of two strings. For example, if we have two strings **S**=" **Cod**" and **T=" ing,"** then the concatenation **ST** would be "**Coding**."

2. Language

A language can be defined as a finite set of strings over some symbols or alphabets.

Operations on Language

The following operations are performed on a language in the lexical analysis phase-

a) Union
b) Concatenation
c) Kleene closure
d) Positive closure

a) Union

Union is one of the most common operations we perform on a set. In terms of languages also, it will hold a similar meaning.

For example If **L = {a, b}** and **S = {c, d}**Then **L ∪ S = {a, b, c, d}**

b) Concatenation

Concatenation links two languages by linking the strings from one language to all the strings of the other language.

The following example shows the operations on strings: Let L={0,1} and S={a,b,c} then

LS= {0a, 0b, 0c, 1a, 1b, 1c }

c) Kleene closure

Kleene closure of a language **L** is denoted by **L***provides a set of all the strings that can be obtained by concatenating **L** zero or more times.

If L = {a, b} then L* = {ε, a, b, aa, bb, aaa, bbb, …}

d) Positive Closure

L+ denotes the Positive closure of a language **L** and provides a set of all the strings that can be obtained by concatenating **L** one or more times.

If L = {a, b} then L+ = {a, b, aa, bb, aaa, bbb, …}

3. Regular Expression

Regular expressions are strings of characters that define a searching pattern with the help of which we can form a language, and each regular expression represents a language.

It is an algebraic formula used to represent a set of strings (a regular language).

A regular expression **r** can denote a language **L(r)** which can be built recursively over the smaller regular expression by following some rules.

Following symbols are used very frequently to write regular expressions

Metacharacter	Name	Matches
.	Dot	Matches any one character.
[…]	Character class	Matches any one character listed.
[^…]	Negated character class	Matches any one character not listed.
?	Question	One allowed, but it is optional.
*	Star	Any number allowed, but all are optional.
+	Plus	At least one required; additional are optional.
|	Alternation	Matches either expression it separates.
^	Caret	Matches the position at the start of the line.
$	Dollar	Matches the position at the end of the line.
{X,Y}	Specified range	X required, max allowed

Operations on Regular expression

Consider the two regular expressions r and s then:

r|s denotes the language L(r) ∪ L(s) → Union

(r) (s) denotes the language L(r) · L(s). → Concatenation

(r)* denotes the language (L(r))*. → Kleene Closure

(r)+ denotes the language (L(r))+ → Positive Closure

The unary operator * has highest precedence and is left associative. Concatenation has second highest precedence and is left associative. | (Union) has lowest precedence and is left associative.

Regular set

A language that can be defined by a regular expression is called a regular set. If two regular expressions r and s denote the same regular set, we say they are equivalent and write r = s.

There are a number of algebraic laws for regular expressions that can be used to manipulate into equivalent forms.

LAW	Description
r|s = s|t	Union is commutative
r|(s|t) = (r|s)|t	Union is associative
r(st) = (rs)t	Concatenation is associative
r(s|t) = rs|rt; (s|t)r = sr|tr	Concatenation distribute over
∈r = r∈ = r	∈ is identity for concatenation
$r^* = (r|\in)^*$	∈ is guaranteed in a closure
r** = r*	* is idempotent

Regular Definitions

Regular definitions are the names given to certain regular expressions and these names are later used for constructing the patterns. Regular definition is a sequence of the form

$d_1 \rightarrow r_1$

$d_2 \rightarrow r_2$

……

$d_n \rightarrow r_n$

1.Each d_i is a distinct name.

2.Each r_i is a regular expression over the alphabet $\Sigma \cup \{d_1, d_2, \ldots, d_{i-1}\}$.

Example: *Identifiers is the set of strings of letters and digits beginning with a letter. Regular definition for this set: (letter, digit, id)*

letter → A | B | …. | Z | a | b | …. | z |

digit → 0 | 1 | …. | 9

id → letter (letter | digit) *

➢ RECOGNITION OF TOKENS

Token recognition is the process of identifying lexemes in the source program and classifying them into their corresponding token categories based on predefined patterns (usually specified using regular expressions).

Tokens are typically specified using **regular expressions** and recognized using **finite automata (FA)**:

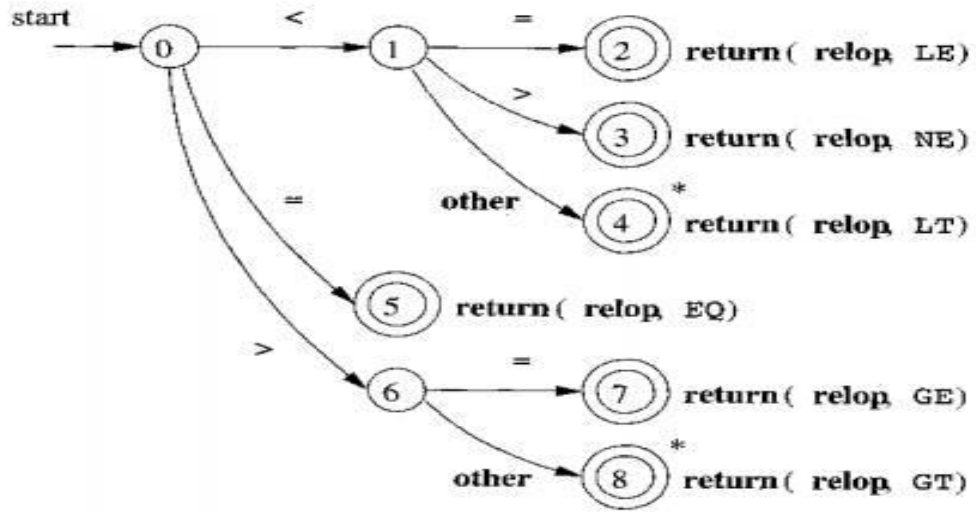
Regular Expressions → Finite Automata → Lexical Analyzer → Token Recognition.

i) A transition diagram that recognizes the lexemes matching the token relop.

Token: `RELOP`

Specification: **`relop → < | <= | > | >= | == | <>`**

Finite Automata for *relational operators* (relop)



The * used at the final state in finite automata for relational operators (like <, >) acts as a **retraction indicator**. It signals the **lexical analyzer** to step backward by one character before returning the recognized token.

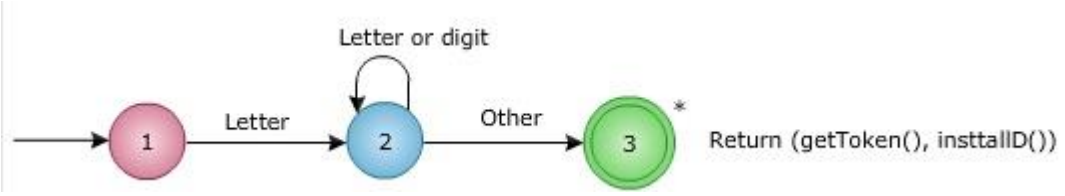
ii). Recognition of Identifiers

Recognizing keywords and identifiers presents a problem. Usually, keywords like if or **then** are reserved (as they are in our running example), so they are not identifiers even though they *look* like identifiers.

Token: `ID`

```
letter          [a-zA-Z_]
digit           [0-9]
id              {letter}({letter}|{digit})*
```

Finite Automata for recognizing *Identifiers*



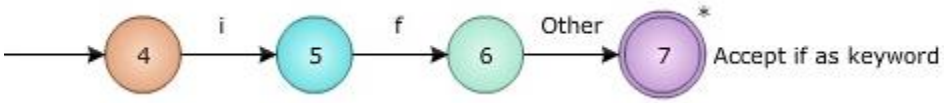
installID() checks if the lexeme is already in the table. If it is not present, the lexeme is installed as an id token. In either case a pointer to the entry is returned.

gettoken() examines the lexeme and returns the token name, either id or a name corresponding to a reserved keyword.

iii**). Recognition of Identifiers**

- If it matches the identifier pattern, the lexer then checks if the recognized string is present in a predefined list of keywords. If it is, it's a keyword; otherwise, it's an identifier. This disambiguation is crucial because keywords often adhere to the identifier pattern.

Finite Automata for recognizing keyword **(if)**

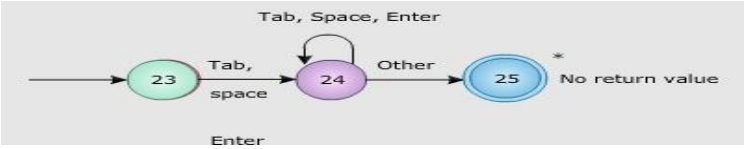


vi**). Recognition of white spaces**

Here, **blank, tab,** and **newline** are abstract symbols that we use to express the ASCII characters of the same names. Token *ws* is different from the other tokens in that, when we recognize it, we do not return it to the parser, but rather restart the lexical analysis from the character that follows the whitespace

Specification : **ws → (blank | tab** | **enter)**$^+$

Finite Automata for recognizing white spaces



v**) Recognition of Numbers**

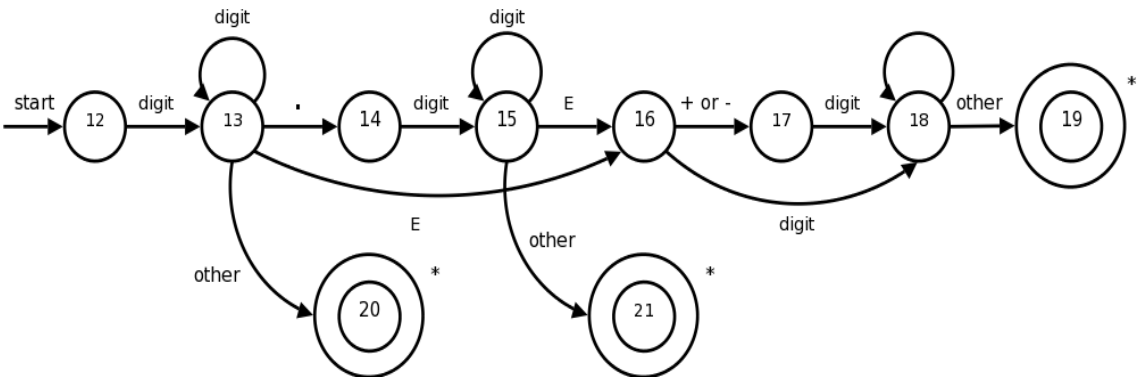
Specification **`num → digit+(.digit+) ? (E(+|-) ? digit +) ?`**

`digit+` represents the integer part of the number.
digit = 0|1|2|3|4|5|6|7|8|9
+ means one or more occurrences

(. digit+)? this part represents the fractional (decimal) part, which may or may not be present.
. represents the decimal point.
digit+ after the decimal point represents one or more digits.
? means optional (zero or one occurrence).

(E (+|−)? digit+)? This part represents the exponent in scientific notation.
E indicates the beginning of the exponent.
(+|−)? means an optional sign (+ or −).
digit+ gives the exponent value.
The entire exponent part is optional.

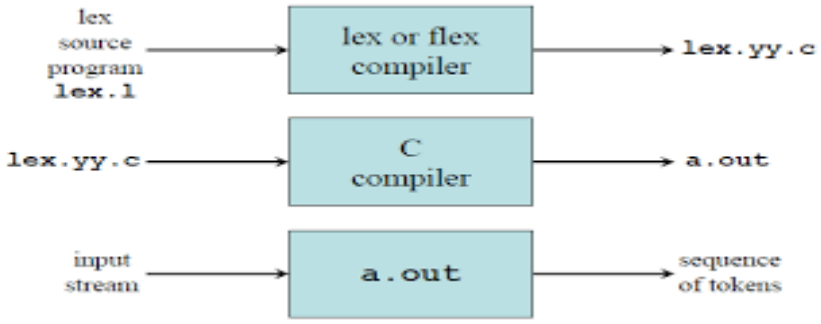
Finite Automata for recognizing Numbers



➢ LEXICAL-ANALYZER GENERATOR LEX

LEX is a lexical analyzer generator, which is a tool that automatically creates a lexical analyzer (also known as a scanner or lexer) from a set of rules specified by the user. These rules, written in a specialized language using regular expressions, define how input text should be broken down into tokens, which are the fundamental units of a programming language. Essentially, LEX takes a description of the desired token patterns and generates C code for a lexical analyzer that can efficiently identify and categorize these patterns in input streams.

The Lex tool is a compiler itself. The Lex compiler transforms the input into input patterns.



Components of a LEX program

A LEX program typically consists of three parts: An initial declaration section, a middle set of translation rules and the last section that consists of other auxiliary procedures.

A double modulus sign separates each section %% acts as a delimiter A program may miss the declaration section but the delimiter is mandatory.

```
Declaration
%%
Translation Rules
%%
Auxiliary Procedures
```

i) Definitions / Declaration

The definitions are at the top of the LEX source file. It includes the regular definitions, the C, directives, and any global variable declarations. The code specific to C is placed within %{%{ and %}%}.

For example:

```
%{
#include <stdio.h>
int globalVariable = 0;
%}
```

ii) Rules

The translation rules each have the form

```
Pattern          { Action }
```

Each pattern is a regular expression, which may use the regular definitions of the declaration section. The actions are fragments of code, typically written in C, although many variants of **Lex** using other languages have been created.

They execute whenever a token in the input stream matches with the grammar.

iii) Auxiliary routines

This section includes functions that may be required in the rules section. Here, a function is written in regular C syntax. Most simple lexical analyzers only require the main() function.

Alternatively, these functions can be compiled separately and loaded with the lexical analyzer.

The **yytext** keyword gives the current lexeme. The generated code is placed into a function called **yylex()**. The **main()** function always calls the **yylex()** function.

Lex program to recognize identifiers and numbers

```
%{
      #include <stdio.h>
%}
letter [a-zA-Z_]
digit [0-9]
%%

{letter}({letter}|{digit})*  { printf("%s is identifier \n", yytext); }
{digit}+              { printf("%s is number \n", yytext); }
.     ;
%%

int main()
{
      printf("Enter input string \n");
      yylex();
      return 0;
}
int yywrap()
{
      return 1;
 }
```

yywrap( )

- yywrap is called by lex when it encounters the end of the current input file.

If yywrap returns 0, lex assumes there is more input to process and continues scanning. If it returns a non-zero value (usually 1), lex considers the input complete and terminates the scanning process.

- By default, yywrap returns 1, indicating the end of input.
- Users can redefine yywrap to handle multiple input files or other input scenarios. For example, if you want to process multiple files, you would redefine yywrap to return 0 until all files are processed, and then return 1.

➢ FINITE AUTOMATA

- An automaton with a finite number of states is called a finite automaton (FA) or finite-state machine (FSM). Finite automata are abstract machines used to recognize patterns
- It takes the string of symbol as input and changes its state accordingly. When the desired symbol is found, then the transition occurs.
- At the time of transition, the automata can either move to the next state or stay in the same state.
- When the input string is processed successfully, and the automata reached its final state, then it will accept otherwise, it is rejected.

- Finite automata (FA) are fundamental in compiler design, particularly for lexical analysis. They act as recognizers for regular languages, enabling the identification of tokens (keywords, operators, etc.) within source code. By defining states and transitions based on input symbols, FA effectively parse the input and determine if it conforms to a predefined pattern.

Types of Finite Automata:

a) Deterministic Finite Automata (DFA):
   For each state and input symbol, there is exactly one transition to another state. In DFA, no null moves are accepted.

b) Non-deterministic Finite Automata (NFA):
   For a state and input symbol, there can be zero, one, or multiple transitions to other states. NFAs can be converted to DFAs, but DFAs are generally preferred for implementation due to their deterministic nature.

Every regular expression and every NFA can be converted to a DFA accepting the same language, because it is the DFA that we really implement or simulate when building lexical analyzers.

Formal Definition of FA

A finite automaton is a collection of 5-tuple (Q, ∑, δ, q0, F), where:

1. Q: finite set of states
2. ∑: finite set of the input symbol
3. q0: initial state
4. F: final state or accept state
5. δ: Transition function

For DFA, δ is the transition function is $\delta: Q \times \sum \rightarrow Q$

For NFA , δ is the transition function is $\delta: Q \times \sum \rightarrow 2^Q$

Representation of FA (DFA or NFA)

There are two preferred notations for describing automata (NFA or DFA)

1. A *transition diagram* which is a graph
2. A *transition table* which is a tabular listing of the function which by implication tells us the set of states and the input alphabet

1 Transition Diagrams

A transition diagram for a DFA A =(Q, ∑, δ, q0, F) is a graph defined as follows

- For each state in Q there is a node
- A transition diagram is a directed graph where:
  - **Nodes**: represent the states of the automaton.
  - **Edges**: (directed arcs) represent transitions between states, labeled with input symbols.
  - An initial state is indicated by an incoming arrow without a source state.
  - **Final states**: are indicated by double circles.

2. Transition Table:

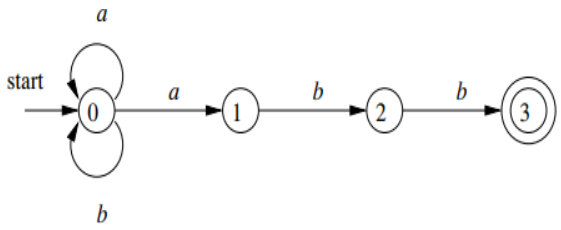
- A transition table is a tabular representation that shows:
  - **Rows**: representing the states of the automaton.
  - **Columns**: representing the input symbols.
  - **Cells**: containing the next state(s) the automaton transitions to when receiving the corresponding input symbol in the current state.

If the transition function has no information about that state-input pair , we put ∅ in the table for the pair

If we are using a transition table to represent a DFA, then each entry is a single state, we may therefore represent this state without the curly braces that we use to form sets.

Example

The transition graph for an NFA recognizing the language of regular expression (a|b)* abb

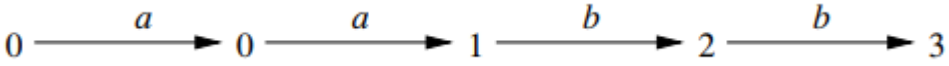


STATE	a	b	ϵ
0	$\{0,1\}$	$\{0\}$	$\emptyset$
1	$\emptyset$	$\{2\}$	$\emptyset$
2	$\emptyset$	$\{3\}$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

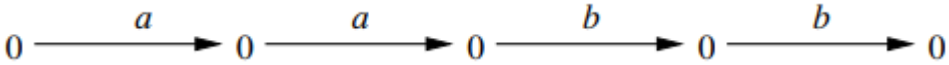
: Transition table for the NFA

The string *aabb* is accepted by the NFA

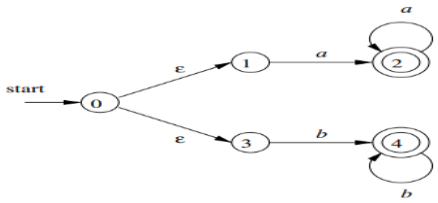
The path labeled by *aabb* from state 0 to state 3 demonstrating this fact is:



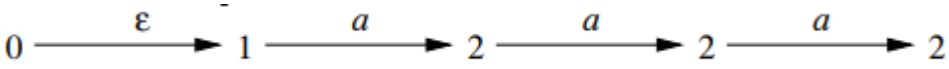
Note that several paths labeled by the same string may lead to different states. For instance, another path is



Example : NFA for language accepting aa*| bb*

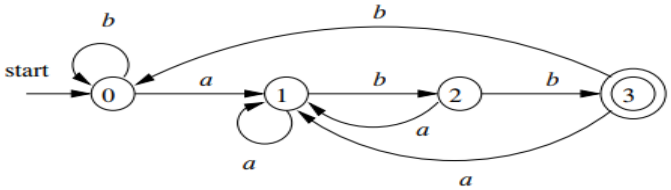


String aaa is accepted because of the path



Example

The transition graph for an DFA recognizing the language of regular expression (a|b)* abb



➢ From Regular Expressions to Automata

The following steps are involved in the construction of DFA from regular expression:

i) Convert RE to NFA using Thomson's rules

ii) Convert NFA to DFA

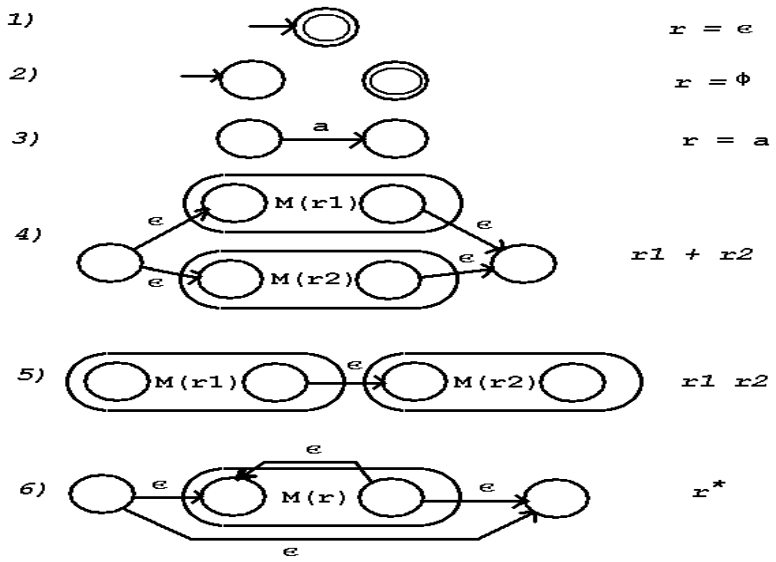
iii) Construct minimized DFA

i)Construction of NFA from a Regular expression

Method

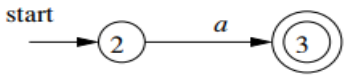
1. Produce an NFA for each RE.
2. Introduce a new start state.
3. Introduce an ε transition from the new start state to the start of each NFA constructed in step 1.
4. When one of the NFAs reaches one of the accepting states, the simulation does **NOT** stop. See below for an explanation.

Remove Null transition from the NFA and convert it into its equivalent DFA.

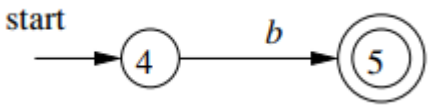


Example : Construction of NFA for Regular expression for (a|b)* abb

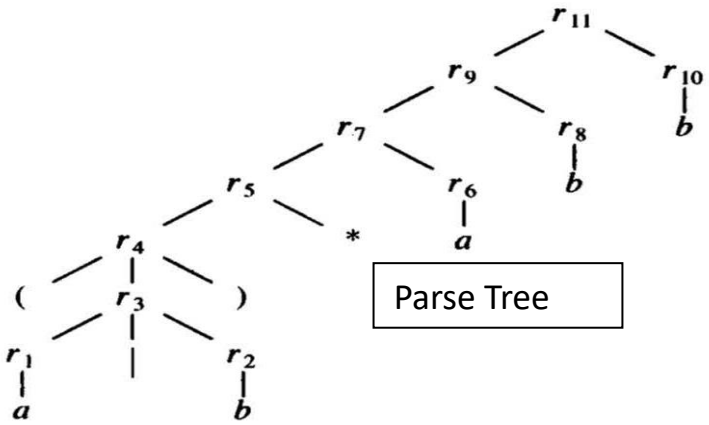
NFA for r1 = a



NFA for r2 = b

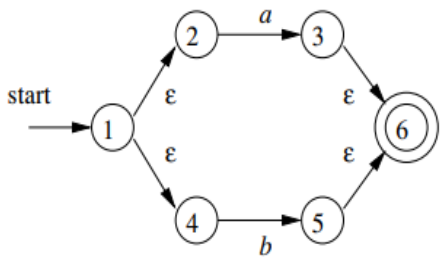


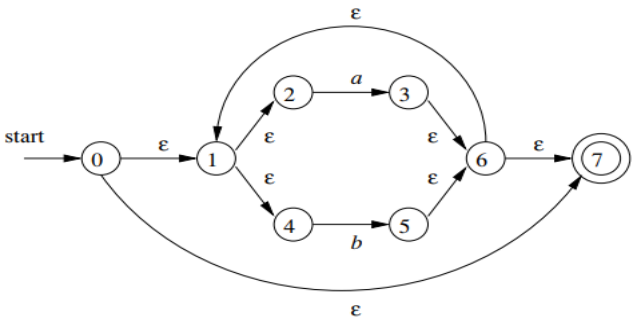
NFA for r3 = a | b



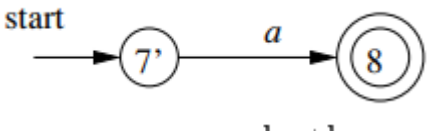
Parse Tree

NFA for r5 = (a | b) *

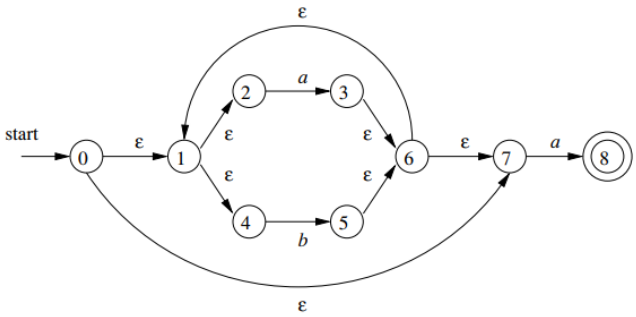




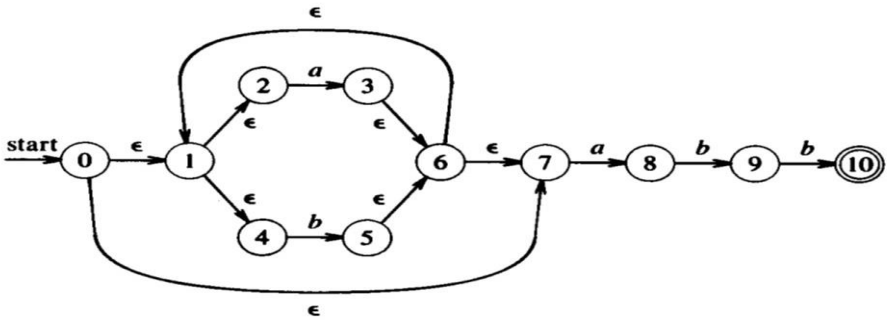
NFA for r6 =a



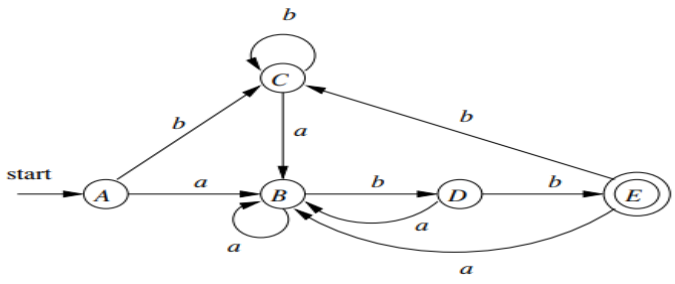
NFA for r7 = (a |b)*a



Finally complete NFA is r11 = (a |b)*abb



Equivalent DFA will be



- **CONVERSION OF NFA TO DFA**

  Every nondeterministic finite automaton (NFA) can be converted to a deterministic finite automaton (DFA).

  Each state in the DFA represents a **set of states** in the NFA.

  The number of states in the resulting DFA may differ from that of the original NFA.

  If the NFA has **N states**, the DFA can have at most $\mathbf{2^N}$ **states**.

A DFA state is accepting if any of its corresponding NFA states is an accepting state.

The following steps are followed to convert a given NFA to a DFA-

Step-01:

Let Q' be a new set of states of the DFA. Q' is null in the starting.

Let T' be a new transition table of the DFA.

Step-02:

Add start state of the NFA to Q'.

- Add transitions of the start state to the transition table T'.
- If start state makes transition to multiple states for some input alphabet, then treat those multiple states as a single state in the DFA.

Step-03:

If any new state is present in the transition table T',

- Add the new state in Q'.
- Add transitions of that state in the transition table T'

Step-04:

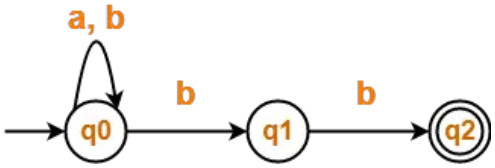
Keep repeating Step-03 until no new state is present in the transition table T'.

Finally, the transition table T' so obtained is the complete transition table of the required DFA.

If "x" was the final state in NFA, then all those states will be the final in DFA where "x" exists.

Example :

Convert the following Non-Deterministic Finite Automata (NFA) to Deterministic Finite Automata (DFA)- NFA for String ending with 'bb'



Solution

Transition table for the given Non-Deterministic Finite Automata (NFA) is-

State / Alphabet	A	B
→**q0**	q0	q0, q1
q1	**Ø**	*q2
***q2**	**Ø**	**Ø**

Step-01:

Let Q' be a new set of states of the Deterministic Finite Automata (DFA).

Let T' be a new transition table of the DFA.

Step-02:

Add transitions of start state q0 to the transition table T'.

State / Alphabet	A	B
→**q0**	q0	{q0, q1}

Step-03:

New state present in state Q' is {q0, q1}.

Add transitions for set of states {q0, q1} to the transition table T'.

State / Alphabet	A	B
→**q0**	q0	{q0, q1}
{q0, q1}	q0	{q0, q1, q2}

Step-04:

New state present in state Q' is {q0, q1, q2}.

Add transitions for set of states {q0, q1, q2} to the transition table T'.

State / Alphabet	A	B
→**q0**	q0	{q0, q1}
{q0, q1}	q0	{q0, q1, q2}
{q0, q1, q2}	q0	{q0, q1, q2}

Step-05:

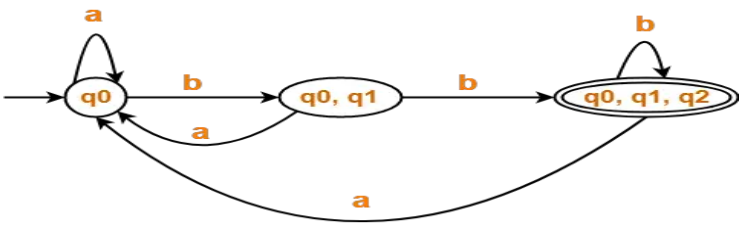
Since no new states are left to be added in the transition table T', so we stop.

States containing q2 as its component are treated as final states of the DFA.

Finally, Transition table for Deterministic Finite Automata (DFA) is-

State / Alphabet	A	B
→**q0**	q0	{q0, q1}
{q0, q1}	q0	*{q0, q1, q2}
***{q0, q1, q2}**	q0	*{q0, q1, q2}

Now, Deterministic Finite Automata (DFA) may be drawn as-



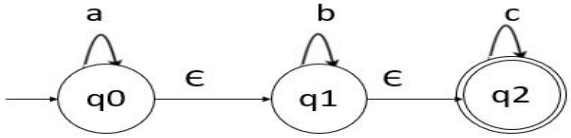
Deterministic Finite Automata (DFA)

Epsilon-Closures :

The ε closure(P) is a set of states which are reachable from state P on ε-transitions including the state itself..

Example

Find ε-closure for the following Non-deterministic finite automata (NFA) with epsilon.



ε-closure (q0) = {q0,q1,q2} self state+ ε-reachable states.

ε-closure (q1) = { q1,q2} q1 is self-state and q2 is a state obtained from q1 with epsilon input.

ε-closure (q2) = {q2}

➢ STEPS FOR CONVERTING NFA WITH ε TO DFA:

Step 1: We will take the ε-closure for the starting state of NFA as a starting state of DFA.

Step 2: Find the states for each input symbol that can be traversed from the present. That means the union of transition value and their closures for each state of NFA present in the current state of DFA.

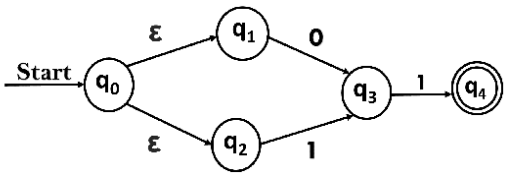
Step 3: If we found a new state, take it as current state and repeat step 2.

Step 4: Repeat Step 2 and Step 3 until there is no new state present in the transition table of DFA.

Step 5: Mark the states of DFA as a final state which contains the final state of NFA.

Example :

Convert the NFA with ε into its equivalent DFA.



Solution:

Let us obtain ε-closure of each state.

1. ε-closure {q0} = {q0, q1, q2}
2. ε-closure {q1} = {q1}
3. ε-closure {q2} = {q2}
4. ε-closure {q3} = {q3}
5. ε-closure {q4} = {q4}

Now, let ε-closure {q0} = {q0, q1, q2} be state A.

Hence

δ'(A, 0) = ε-closure {δ((q0, q1, q2), 0) }

= ε-closure {δ(q0, 0) ∪ δ(q1, 0) ∪ δ(q2, 0) }

= ε-closure {q3}

= {q3} **call it as state B .**

δ'(A, 1) = ε-closure {δ((q0, q1, q2), 1) }

= ε-closure {δ((q0, 1) ∪ δ(q1, 1) ∪ δ(q2, 1) }

= ε-closure {q3} = {q3} = B.

Now,

δ'(B, 0) = ε-closure {δ(q3, 0) } = φ

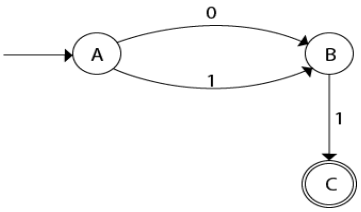
δ'(B, 1) = ε-closure {δ(q3, 1) }

= ε-closure {q4} = {q4} **i.e. state C**

For state C:

1. δ'(C, 0) = ε-closure {δ(q4, 0) } = φ
2. δ'(C, 1) = ε-closure {δ(q4, 1) } = φ

The DFA will be



➢ DESIGN OF A LEXICAL-ANALYZER GENERATOR

➢ THE STRUCTURE OF THE GENERATED ANALYZER

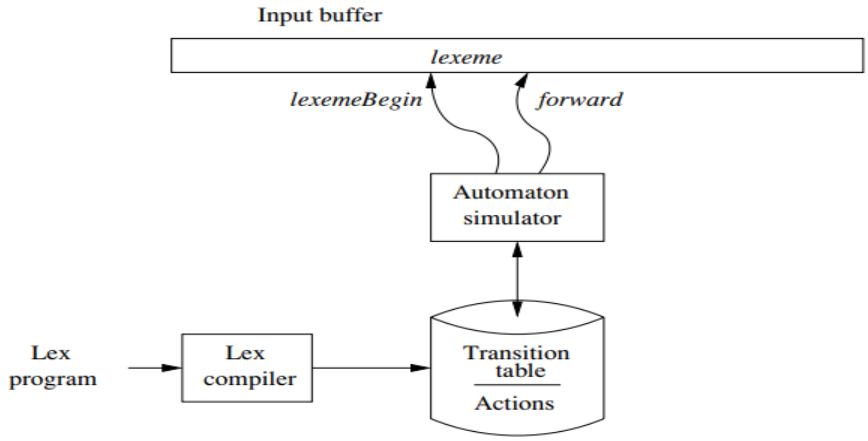
A generated analyzer is a software tool that automatically constructs a lexical analyzer from a program in the lex language.

lexer-generators do not draw pictures; instead they use the equivalent transition tables.

The lex compiler generates a transition table for a finite automaton from the regular expressions in the lex specification.

The lexical analyzer consists of finite automaton simulator that uses the transition table to look for the regular expression patterns in the input buffer.

The architecture of a lexical analyzer generated by *Lex*



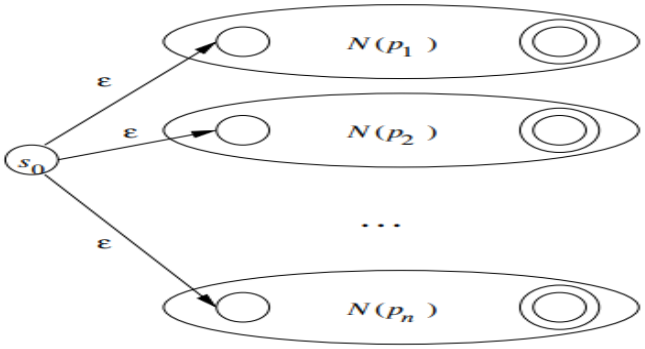
To construct the automaton, we begin by taking each regular-expression pattern in the Lex program and converting it, using Algorithm, to an NFA.

Lex input will contain several REs since it wishes to recognize several different tokens. The solution is to

1. Produce an NFA for each RE.
2. Introduce a new start state.
3. Introduce an ε transition from the new start state to the start of each NFA constructed in step 1.

We need a single automaton that will recognize lexemes matching any of the patterns in the program, so we combine all the NFA's into one by introducing a new start state with -transitions to each of the start states of the NFA's Ni for pattern P_i.

NFA constructed from a Lex Program



We look backwards in the sequence of sets of states, until we find a set that includes one or more accepting states. If there are several accepting states in that set, pick the one associated with the earliest pattern *pi* in the list from the **Lex** program. Move the *forward* pointer back to the end of the lexeme, and perform the action *Ai* associated with pattern *pi.*

Pattern Matching Based on NFAs

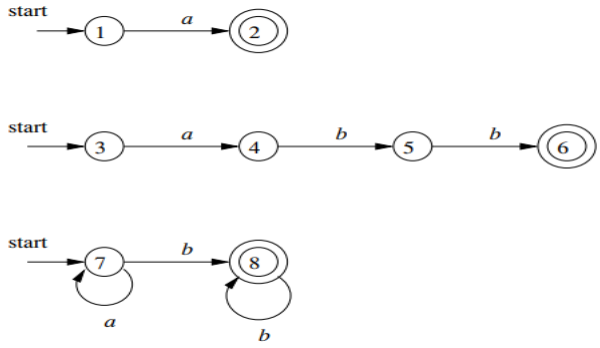
The simulator starts reading characters and calculates the set of states it is at.

At some point the input character does not lead to any state or we have reached the eof.

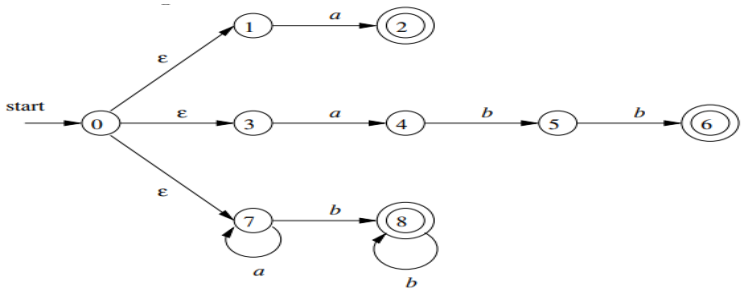
Since we wish to find the *longest* lexeme matching the pattern we proceed backwards from the current point (where there was no state) until we reach an accepting state (i.e., the set of N-states contains an accepting N-state).

Each accepting N-state corresponds to a matched pattern. The lex rule is that if a lexeme matches multiple patterns we choose the pattern listed first in the lex-program.

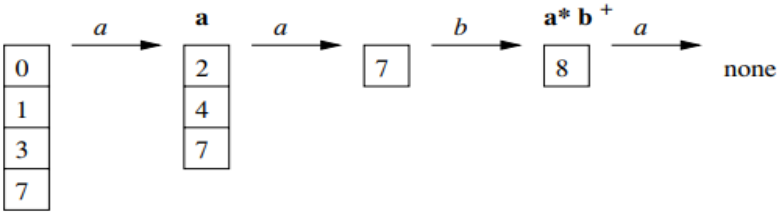
Example For example NFA s for a, abb , a*b+



Combined NFA



Sequence of sets of states entered when processing input 'aaba'



Lex uses these rules:

1.Choose the longest match.

2 If two or more rules match the same longest text, choose the rule that appears first in the Lex file

DFA's for Lexical Analyzers

Another architecture, resembling the output of Lex is to convert the NFA for all the patterns into an equivalent DFA, using the subset construction of Algorithm . Within each DFA state, if there are one or more accepting NFA states, determine the the first pattern whose accepting state is represented, and make that pattern the output of the DFA state.

when we construct a DFA for use in a lexical analyzer, it is important that we treat the dead state differently, since we must know when there is no longer any possibility of recognizing a longer lexeme. Thus, we suggest always omitting transitions to the dead state and eliminating the dead state itself.

➢ OPTIMIZATION OF DFA-BASED PATTERN MATCHERS

Optimizing DFA-based pattern matchers in compiler design involves techniques to reduce the *size* and *complexity* of the Deterministic Finite Automaton (DFA) without affecting its ability to accurately match patterns.

This leads to *faster* execution and reduced memory consumption.

This can be achieved through various techniques, such as state minimization and transition compression.

To construct a DFA directly from a regular expression, we construct its syntax tree and then compute four functions: *nullable, firstpos, lastpos,* and *followpos,* defined as follows. Each definition refers to the syntax tree for a particular augmented regular expression (r) # .

1. ***nullable****(n)* is true for a syntax-tree node **n** if and only if the subexpression represented by *n* has € (epsilon) in its language. That is, the subexpression can be "made null" or the empty string, even though there may be other strings it can represent as well.

2. ***firstpos****(n)* is the set of positions in the subtree rooted at **n** that correspond to the first symbol of at least one string in the language of the subexpression rooted at **n.**

3. ***lastpos****(n)* is the set of positions in the subtree rooted at *n* that correspond to the last symbol of at least one string in the language of the subexpression rooted at **n.**

4. **followpos**(p), for a position p, is the set of positions q in the entire syntax tree such that there is some string x = axa2 ••• an in L ((r) #) such that for some i, there is a way to explain the membership of x in L ((r) #) by matching to position p of the syntax tree and ai+i to position g.

Node Type	nullable	firstpos	lastpos
Leaf node (symbol a)	False	{position}	{position}
Leaf node (ε)	True	∅	∅
Union (|)	nullable(c_1) OR nullable(c_2)	firstpos(c_1) ∪ firstpos(c_2)	lastpos(c_1) ∪ lastpos(c_2)
Concatenation(.)	nullable(c_1) AND nullable(c_2)	If nullable(c_1), then firstpos(c_1) ∪ firstpos(c_2) else firstpos(c_1)	If nullable(c_2), then lastpos(c_1) ∪ lastpos(c_2) else lastpos(c_2)
Kleene Star (*)	True	firstpos(c_1)	lastpos(c_1)

Two rules define the function followpos

1. If n is concatenation-node with left child c_1 and right child c_2,and i is a position in lastpos(c_1), then all positions in firstpos(c_2) are in followpos(i).
2. If n is a star-node, and i is a position in lastpos(n), then all positions in firstpos(n) are in followpos(i).

If firstpos and lastpos have been computed for each node, followpos of each position can be computed by making one depth-first traversal of the syntax tree.

Converting A Regular Expression Directly To A Dfa

1.Firstly, we construct the augmented regular expression for the given expression. By concatenating a unique right-end marker '#' to a regular expression r. This symbol helps identify the final state(s) in the resulting DFA. Example: For (a|b)*abb, the augmented expression becomes (a|b)*abb#.

2.Construct a syntax tree T from the augmented regular expression (r) # .

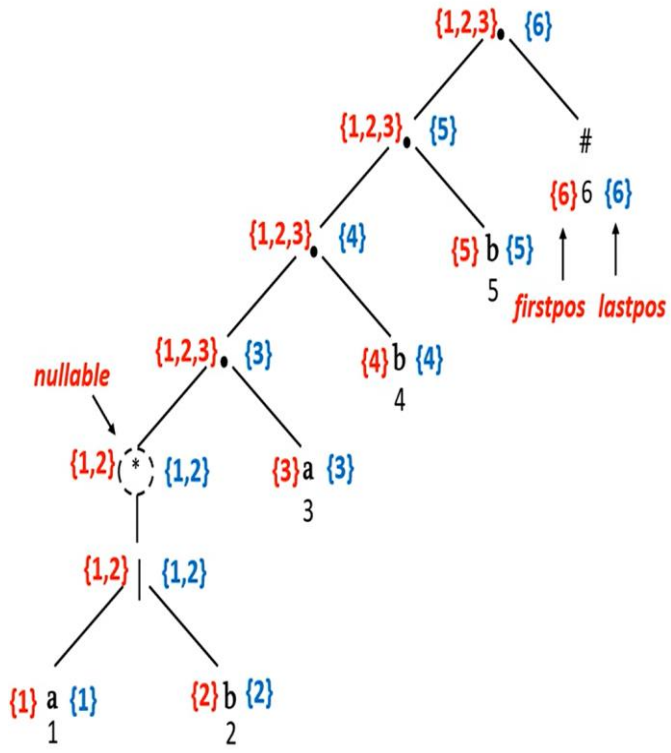
3.Assign a unique position number to each leaf node representing an input symbol or the end-marker in the syntax tree.

4.Compute nullable, firstpos, and lastpos for each node:

5. Compute followpos for each position.

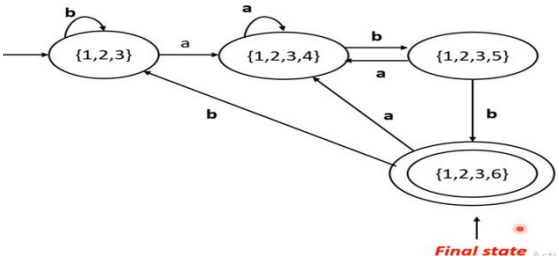
6. Construct the DFA:

- The initial state of the DFA is firstpos(root_node_of_syntax_tree).
- For each state (set of positions) S in the DFA and for each input symbol a:
  - Calculate the new state S' by taking the union of followpos(p) for all positions p in S where the symbol at position p is a.
  - Add a transition from S to S' on input a.
- Any state containing the position of the '#' symbol is a final state.
- Repeat until no new states are generated.



(a | b)*. a . b . b. #
1 2 3 4 5 6

Node name	Node	followpos
a	1	{1,2,3}
b	2	{1,2,3}
a	3	{4}
b	4	{5}
b	5	{6}
#	6	-



➢ MINIMZATION OF FINITE AUTOMATA

Minimization of DFA means reducing the number of states from given FA. Thus, we get the FSM(finite state machine) with redundant states after minimizing the FSM.

2 approaches are there

1. Equivalence method or partition method
2. Table filling method or myhill-nerode method

Method 1 : Equivalence method or partition method

We have to follow the various steps to minimize the DFA. These are as follows:

Step **1:**

Eliminate all the dead states and inaccessible states from the given DFA (if any).

Step **2:**

- Draw a state transition table for the given DFA.
- Transition table shows the transition of all states on all input symbols in Σ.

Step **3**

Now, start applying equivalence theorem.

- Take a counter variable k and initialize it with value 0.

- Divide Q (set of states) into two sets such that one set contains all the non-final states and other set contains all the final states.
- This partition is called P_0.

Step **4:**

- Increment k by 1.
- Find P_k by partitioning the different sets of P_{k-1} .
- In each set of P_{k-1} , consider all the possible pair of states within each set and if the two states are distinguishable, partition the set into different sets in P_k.

Two states q1 and q2 are distinguishable in partition Pk for any input symbol 'a', if δ (q1, a) and δ (q2, a) are in different sets in partition Pk-1.

Step 5:

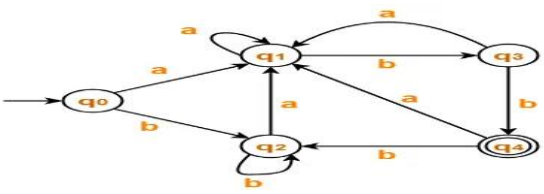
- Repeat step-04 until no change in partition occurs.
- In other words, when you find $P_k = P_{k-1}$, stop.

Step-**06:**

- All those states which belong to the same set are equivalent.
- The equivalent states are merged to form a single state in the minimal DFA.

Number of states in Minimal DFA = Number of sets in P_k

Example : Minimize given DFA



Step1 : The given DFA contains no dead states and inaccessible states.

Step2 : Transition table for DFA

	A	**B**
→**q0**	q1	q2
q1	q1	q3
q2	q1	q2
q3	q1	*q4
***q4**	q1	q2

Step3 Now using Equivalence Theorem, we have-

$P_0 = \{ q_0 , q_1 , q_2 , q_3 \} \{ q_4 \}$

$P_1 = \{ q_0 , q_1 , q_2 \} \{ q_3 \} \{ q_4 \}$

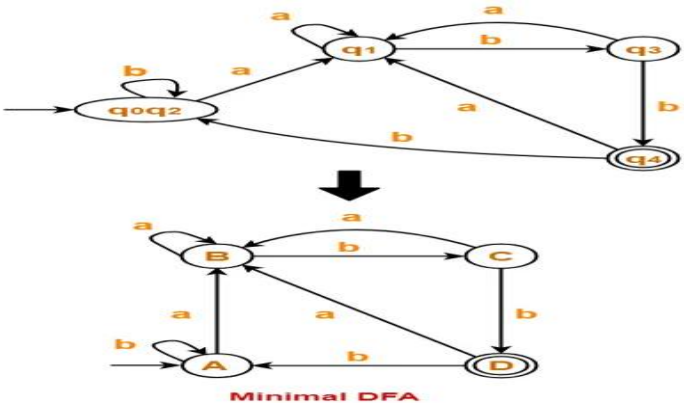
$P_2 = \{ q_0 , q_2 \} \{ q_1 \} \{ q_3 \} \{ q_4 \}$

$P_3 = \{ q_0 , q_2 \} \{ q_1 \} \{ q_3 \} \{ q_4 \}$

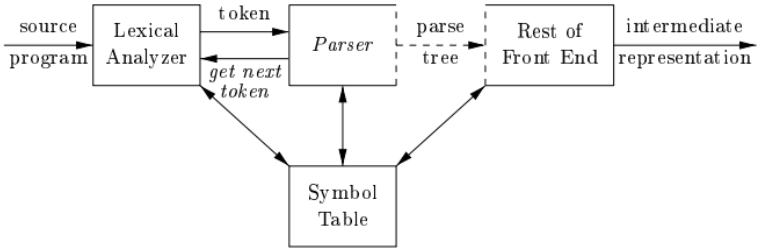
Since $P_3 = P_2$, so we stop.

From P_3, we infer that states q_0 and q_2 are equivalent and can be merged together.

So, Our minimal DFA is-



➢ THE ROLE OF PARSER



The parser or syntactic analyser obtains a string of tokens from the lexical analyser and verifies that the string can be generated by the grammar for the source language.

It reports any syntax errors in the program. It also recovers from commonly occurring errors so that it can continue processing its input.

1. It verifies the structure generated by the tokens based on the grammar.
2. It constructs the parse tree.
3. It reports the errors.
4. It performs error recovery.
5. Parser helps to construct intermediate code.

Example for Syntax Errors

Unbalanced or missing parenthesis , Missing operators , Error in structure Missing ;

Issues **:**

Parser cannot detect errors such as:

1. Variable re-declaration
2. Variable initialization before use
3. Data type mismatch for an operation.

The above issues are handled by Semantic Analysis phase.

Types of Parser

- *Top down parsers* Top down parsers construct parse tree from root to leaves.
- *Bottom up parsers* Bottom up parsers construct parse tree from leaves to root.

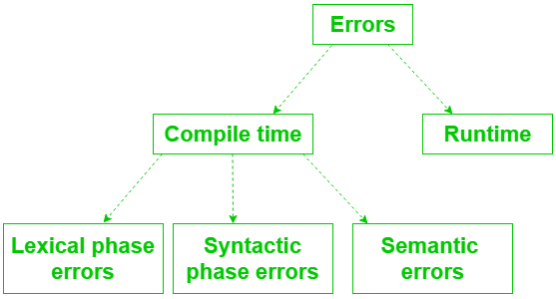
➢ COMMON PROGRAMMING ERRORS

There are two types of errors: run-time errors and compile-time errors:

a) Run-time Error

A run-time error is one that occurs during the execution of a program and is usually caused by incorrect system parameters or invalid input data.

Examples can be a lack of memory to run an application, a memory conflict with another program, and a logical error. Logic errors occur when code is executed and does not produce the desired result.



Errors in the programmer's reasoning or intent, such as using an assignment operator (=) instead of a comparison operator (==) in C, leading to incorrect program behaviour despite being syntactically correct.

b) Compile-time errors

Compile-time errors increase during the compilation process before the program is executed. An example of this is a syntax error or a missing file reference that prevents the program from successfully compiling.

Compile-time errors are further divided into three categories:

- Lexical errors:

Mistakes in the spelling of identifiers, keywords, or operators, or missing quotes around strings.

- Syntactic errors:

Violations of the language's grammar rules, like misplaced semicolons, extra or missing braces, or a case statement without an enclosing switch.

- Semantic errors:

Type mismatches between operators and operands, for example, a Java method returning a value when its return type is void

➢ **ERROR HANDLING:**

Programs can contain errors at many different levels. For example: • Lexical, such as misspelling an identifier, keyword or operators

Functions of error handler:

- It should report the presence of errors clearly and accurately.
- It should recover from each error quickly enough to be able to detect subsequent errors.
- It should not significantly slow down the processing of correct programs.

➢ **ERROR-RECOVERY STRATEGIES**

Once an error is detected, the parser should recover from the error. The following recovery strategies are

Panic-mode:

On discovering an error, the parser discards input symbols one at a time until one of a designated set of synchronizing tokens is found. The synchronizing tokens are usually delimiters, such as semicolon or}, whose role in the source program is clear and unambiguous

Phrase-level:

When an error is identified, a parser may perform local correction on the remaining input; that is, it may replace a prefix of the remaining input by some string that allows the parser to continue. A typical local correction is to replace a comma by a semicolon, delete an extraneous semicolon, or insert a missing semicolon. Phrase-level replacement has been used in several error-repairing compilers, as it can correct any input string. Its major drawback is the difficulty it has in coping with situations in which the actual error has occurred before the point of detection

Error-productions:

The common errors that might be encountered are anticipated and augment the grammar for the language at hand with productions that generate the erroneous constructs.

Global-correction:

A compiler need to make few changes as possible in processing an incorrect input string. There are algorithms for choosing a minimal sequence of changes to obtain a globally least-cost correction. Given an incorrect input string x and grammar G, these algorithms will find a parse tree for a related string y, such that the number of insertions, deletions, and changes of tokens required to transform x into y is as small as possible. These methods are in general too costly to implement in terms of time and space.

➢ **GRAMMAR**

Grammar in automata theory is a standard way of representing the language. The grammar contains a set of production rules that make up the language strings.

The set of all possible strings that can be derived from grammar is known as the language of that grammar. The symbol “G” mostly denotes grammar

Grammars are used to systematically describe the syntax of programming language constructs like expressions and statements.

➢ CONTEXT-FREE GRAMMARS

Context-free grammars (CFGs) are used to describe context-free languages. A context-free grammar is a set of recursive rules used to generate patterns of strings. *A context-free grammar can describe all regular languages and more, but they cannot describe all possible languages*.

The Formal Definition of a Context-Free Grammar

A Grammar is a 4-tuple such that- G = (V , T , P , S) where

- V = Finite non-empty set of non-terminal symbols
- T = Finite set of terminal symbols
- P = Finite non-empty set of production rules
- S = Start symbol

V – Set of Variables (Non-terminals)

- Symbols that can be replaced by other symbols and help define the language generated by the grammar. Also called **non-terminal symbols**.
- Example: if grammar is S→aA A → bA | c then V={S,A}

2. **T** – Set of Terminals
   - Symbols that appear in the final strings of the language. They cannot be replaced further. The terminals are the keywords like if , else and the symbols
   - Example: if grammar is S→aA A → bA | c T={a, b, c}
3. **P** – Set of Production Rules
   - Rules that describe how variables are replaced.
   - General form: $A \rightarrow \alpha$ Where $A \in V$ and $\alpha \in (V \cup T)*$ consisting of zero or more terminals and non-terminals
4. **S** – Start Symbol
   - The non-terminal from which the derivation begins. $S \in V$

EXAMPLE: The grammar with the following productions defines simple arithmetic expression:

expression → *expression* + *term*
expression → *expression* − *term*
expression → *term*
term → *term* * *factor*
term → *term* / *factor*
term → *factor*
factor → (*expression*)
factor → **id**

In this grammar,

Terminals : id, + , - , * , / , (,)
Nonterminals : expression, term, factor
Start symbol : expression

➢ DERIVATIONS

Derivation is generation of string from grammar. Derivation is a process of generating a string by repeatedly applying production rules, starting from start symbol until only terminals remain.
We can use CFG derivation to find out

- Which string is produced by grammar
- Specific string is generated by G or not

To create a string from a context-free grammar, follow these steps:

- Begin the string with a start symbol
- Apply one of the production rules to the start symbol on the left-hand side by replacing the start symbol with the right-hand side of the production.
- Repeat the process of selecting nonterminal symbols in the string, and replacing them with the right-hand side of some corresponding production, until all non-terminals have been replaced by terminal symbols. Note, it could be that not all production rules are used.

➢ SENTENTIAL FORMS

A sentential form is a string consisting of terminals, non-terminals, or both, derived from the start symbol (S) using one or more production rules.

- If S→lm* α, then α is a left sentential forms, and if S→rm * α, then α is a right sentential form.
- A sentential form containing only terminals is called a **sentence**.
- Example: Consider the following grammar for arithmetic expressions:

```
E →→ E+E | E*E | (E) | - E | id
```

To generate a valid string **- (id+id)** from the grammar the steps are

1. E → - E
2. E → - (E)
3. E → - (E+E)
4. E → - (id+E)
5. E → - (id+id)

In the above derivation,

- E is the start symbol.
- -(id+id) is the required sentence (only terminals).
- Strings such as E, -E, -(E), . . . are called **sentinel** forms.

➢ TYPES OF DERIVATIONS

There are two types of derivation:

1. Leftmost Derivation ⇒lm : Here leftmost symbol (variable) is replaced first.
2. Rightmost Derivation ⇒rm: Here rightmost symbol is replaced first

1) Leftmost Derivation

We do a leftmost derivation of a string from a grammar by

1. Begin with a string consisting only of the start symbol.
2. Pick *the leftmost* variable occurrence in the current string.
3. Pick any production that has that variable on the left of the →.
4. Replace the chosen occurrence of that variable by the right-hand side of the chosen production.
5. Repeat steps 2-4 until the string contains only terminals

2) Rightmost Derivation

We do a *rightmost derivation* of a string from a grammar by

1. Begin with a string consisting only of the start symbol.
2. Pick *the rightmost* variable occurrence in the current string.
3. Pick any production that has that variable on the left of the →.
4. Replace the chosen occurrence of that variable by the right-hand side of the chosen production.
5. Repeat steps 2-4 until the string contains only terminals

Example

Consider the G,

$E \rightarrow E + E \mid E * E \mid (E) \mid - E \mid \mathbf{id}$

Derive the string **id + id * id** using leftmost derivation and rightmost derivation.

$$E \overset{lmd}{\Rightarrow} E + E \overset{lmd}{\Rightarrow} \mathbf{id} + E \overset{lmd}{\Rightarrow} \mathbf{id} + E * E \overset{lmd}{\Rightarrow} \mathbf{id} + \mathbf{id} * E \overset{lmd}{\Rightarrow} \mathbf{id} + \mathbf{id} * \mathbf{id}$$

(a)

$$E \overset{rmd}{\Rightarrow} E + E \overset{rmd}{\Rightarrow} E + E * E \overset{rmd}{\Rightarrow} E + E * \mathbf{id} \overset{rmd}{\Rightarrow} E + \mathbf{id} * \mathbf{id} \overset{rmd}{\Rightarrow} \mathbf{id} + \mathbf{id} * \mathbf{id}$$

(b)

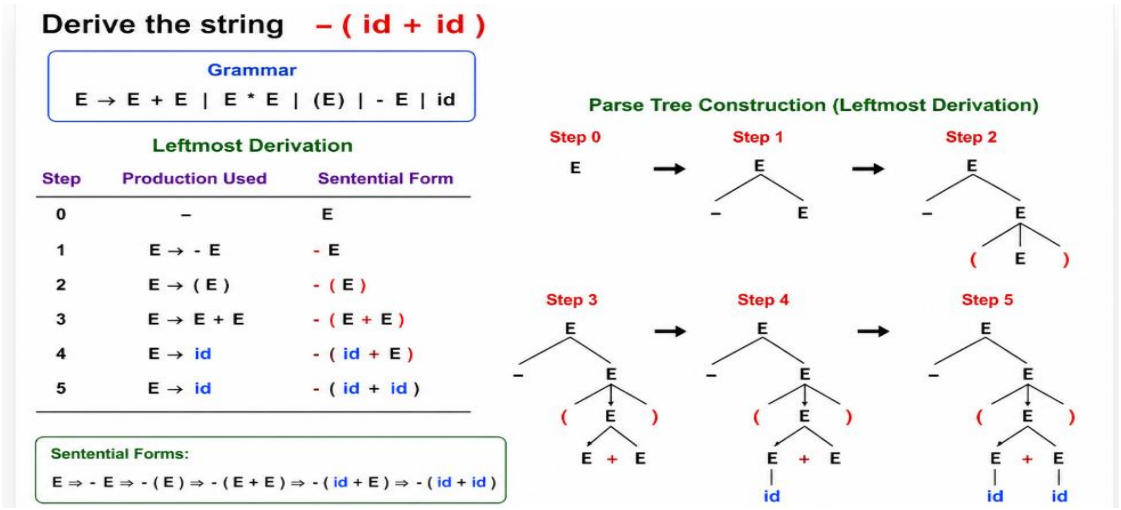
➢ Derivation Tree / Parse Tree:

Parse tree is a tree/ graphical representation of strings of terminals using the productions defined by the grammar. A parse tree pictorially shows how the start symbol of a grammar derives a string in the language. In a parse tree:

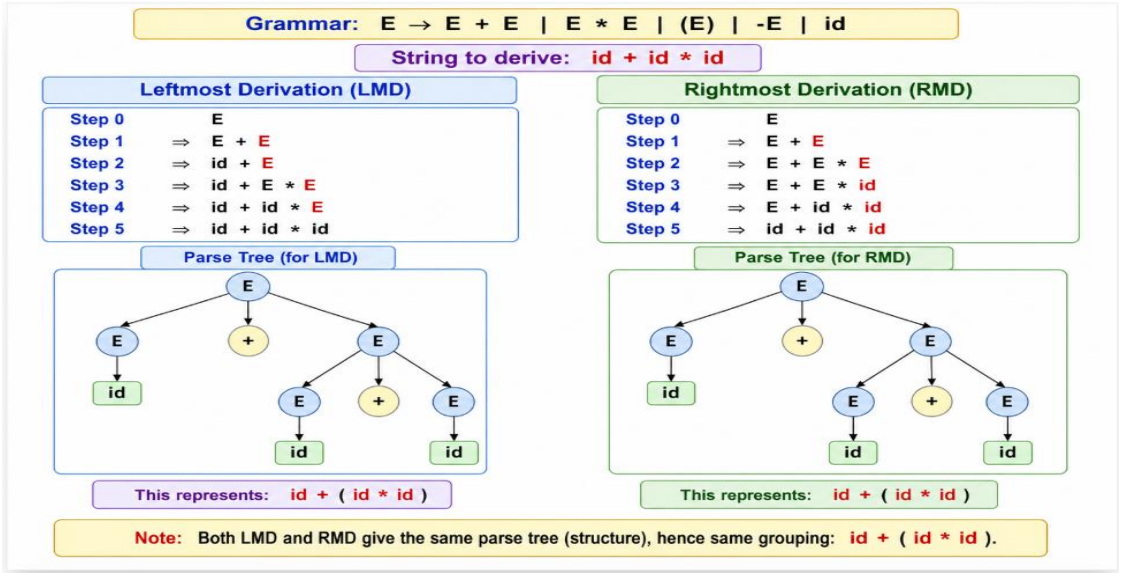
- Inner nodes of a parse tree are non-terminal symbols.
- The leaves of a parse tree are terminal symbols.
- A parse tree can be seen as a graphical representation of a derivation

Yield of tree:

Yield of a parse tree is the sequence of terminal symbols (tokens) at the leaf nodes, read from left to right. The yield is the input string that the parse tree represents.



Example

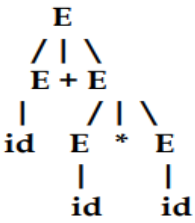


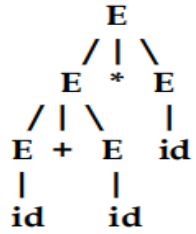
➢ AMBIGUITY

- A Grammar G = (V, T, P and S) is said to be ambiguous if there is a string w ∈ L (G) for which we can derive two or more distinct derivation tree rooted at S and yielding w.
- In other words, a grammar is ambiguous if it can produce more than one leftmost or more than one rightmost derivation for the same string in the language of the grammar.
- If the grammar is not ambiguous then we call it **unambiguous** grammar.
- No method can automatically detect and remove the ambiguity, but we can remove the ambiguity by re-writing the whole grammar without ambiguity.

EXAMPLE Consider the following Grammar **E → E + E | E * E | - E | (E) | id**
And the string (Sentence) is id+ id * id.

1st Leftmost Derivation	2nd Leftmost Derivation
E → E + E → id + E → id + E * E → id + id * E → id + id * id	E → E * E → E + E * E → id + id *E →id + id * E →id + id * id





➢ LEXICAL ANALYSIS VS SYNTACTIC (SYNTAX) ANALYSIS

Lexical Analysis	Syntactic (Syntax) Analysis
It is the first phase of the compiler.	It is the second phase of the compiler.
It reads the source program as a stream of characters.	It reads the tokens generated by the lexical analyzer.
It groups characters into tokens such as identifiers, keywords, operators, and constants.	It checks whether the sequence of tokens follows the grammar rules of the programming language.
The output is a stream of tokens.	The output is a parse tree or syntax tree.
It uses Regular Expressions (RE) and Finite Automata (FA) for token recognition.	It uses Context-Free Grammar (CFG) and parsing techniques (LL, LR, etc.).
It removes white spaces, tabs, newlines, and comments.	It ignores white spaces because they are already handled by the lexical analyzer.
It detects lexical errors such as invalid characters or malformed tokens.	It detects syntax errors such as missing semicolons, unmatched parentheses, or incorrect statement structure.
It creates entries in the symbol table for identifiers.	It uses the symbol table information but does not create tokens.
It is also called a scanner or tokenizer.	It is also called a parser.
Example input: `sum = a + 10;` (character stream).	Example input: `<id> <=> <id> <+> <num> <;>` (token stream).
Software tools : LEX, FLEX, JFLEX	YACC, BISON, BYACC

➢ LEFT RECURSION

There are two types of left recursion in compiler design:

1. Direct left recursion
2. Indirect left recursion

1. Direct left recursion

This is the most straightforward form, in which a non-terminal refers to itself directly in its production rules.

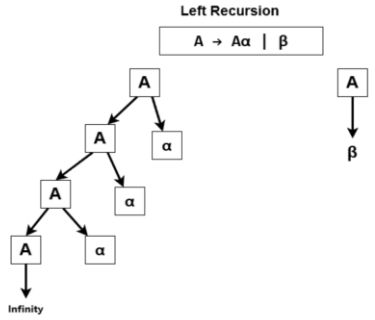
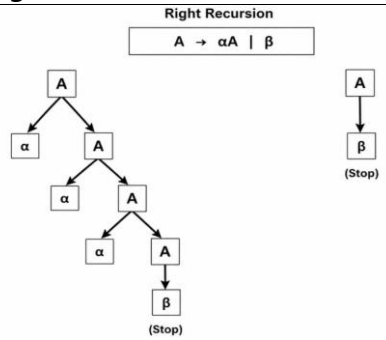
Example : $A \rightarrow A\alpha \mid \beta$

2. Indirect left recursion

It happens when a series of non-terminals refer to each other in a way that eventually leads back to the first non-terminal. .

Example : $A \Rightarrow B\,x$ $B \Rightarrow C\,y$ $C \Rightarrow A\,z$

Left Recursion Vs Right Recursion

Left Recursion	***Right Recursion***
 String generated is βα*	 String generated is α*β

Procedure representation
A()
{
A() ;
α ;
}
	Procedure representation
	A()
	{
	α ;
	A() ;
	}

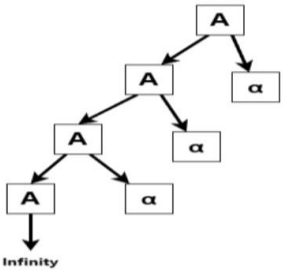
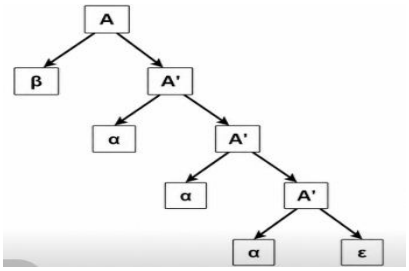
Problems with left recursion in compiler design

- The problem with left recursion is that if left recursion is present in the grammar, then it gives rise to ambiguity.
- And also during top-down parsing in the syntax analysis, there is a possibility that the grammar will create an infinite loop. Top-down parsing uses left most derivation.
- We can use left factoring to remove left recursion from our grammar.

➢ **HOW TO ELIMINATE LEFT RECURSION?**

Algorithm

1. **Identify Left Recursion**: Left recursion often appears as a non-terminal symbol that refers to itself at the start of one of its outputs.
2. **Transform Left-Recursive Rules**: Create new non-terminals for each left-recursive rule to eliminate left recursion. As a rule A -> Aα | β, transform it into A -> βA', where A' represents the remaining alternatives and is defined as A' -> αA' | 'ε' (epsilon).
3. **Update Original Rule**: Replace the existing left-recursive rule with the new one.
4. **Handle Indirect Left Recursion**: If your grammar has indirect left recursion, ensure that the transformation is applied repeatedly until all left recursion is removed.

With Left Recursion	**After Eliminating Left Recursion**
A → A α | β  String generated is βα*	A→ βA' A' → αA' | €  String generated is βα*

Example : Eliminate left recursion for S→ SX|SSB|XS|a X → Xb|Sa

Solution:

S→ XSS′ | aS'

S′ → XS′ | SbS′ | ε

X → SaX′

X′ → bX′ | ε

With Left Recursion	After Eliminating Left Recursion
(i) A → A α | β	A→ βA' A' → αA' | €
(ii) A → A α_1 | A α_2 | A α_3 | β	A→ βA' A' → α_1A' | α_2A'| α_3A'| €
(iii) A → A α | β_1| β_2| β_3	A → β_1A' | β_2A' | β_3A' A' → αA' | €
(iv) A → A α_1 | A α_2 |… | A α_3 | β_1| β_2| …	A → β_1A' | β_2A' | … A' → α_1A' | α_2A'| … | €

Example : Consider the following grammar, eliminate left recursion for S → (L) / a L → L , S / S

Solution- The grammar after eliminating left recursion is-

S → (L) / a

L → SL′

L′ → ,SL′ / ∈

Example : Eliminate left recursion S→ AA|0 A→ SS | 1

Solution: Substitute S productions in A → **S**S | 1

S → AA | 0

A → (**AA**|**0**)s | 1

A → AAS | 0S |1

NOW remove left recursion

S → AA |0

A → 0SA' | 1A'

A' → ASA' | ε

➢ GRAMMAR WITH COMMON PREFIXES

If RHS of more than one production starts with the same symbol, then such a grammar is called as Grammar With Common Prefixes

Example: A → $\alpha\beta_1$ | $\alpha\beta_2$ | $\alpha\beta_3$

- This kind of grammar creates a problematic situation for Top down parsers.
- Top down parsers can not decide which production must be chosen to parse the string in hand.

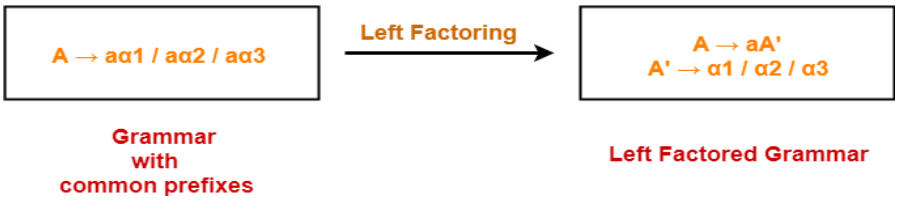
To remove this confusion, we use left factoring.

➢ LEFT FACTORING

- Left factoring is a grammar transformation technique used in compiler design to convert a grammar with common prefixes into an equivalent grammar suitable for top-down parsing.
- Top-down parsers process input by scanning symbols from left to right and matching them against the grammar rules. If two or more rules start with the same symbols, the parser can't immediately decide which one to follow. This results in backtracking or even parsing errors, making the process less efficient.
- Left factoring removes this ambiguity by isolating the common parts of rules, reducing the need for the parser to backtrack.
- With left factoring, we get a clear, step-by-step process that the parser can follow smoothly.

In left factoring,

- We make one production for each common prefixes.
- The common prefix may be a terminal or a non-terminal or a combination of both.
- Rest of the derivation is added by new productions.



The grammar obtained after the process of left factoring is called as **Left Factored Grammar**.

Left factoring improves grammar clarity and efficiency, while left recursion can cause issues for top-down parsers.

Example : Do left factoring in the following grammar

A → aAB | aBc | aAc

After Left factoring

A → aA' A' → AB | Bc | Ac

Again, this is a grammar with common prefixes. So Left factoring once again

A → aA' A' → AD |Bc D → B | c

This is a left factored grammar.

Example: Do left factoring in the following grammar-

S → bSSaaS / bSSaSb / bSb / a

After Left factoring,

S → bSS' | a S' → SaaS | SaSb | b

Again, this is a grammar with common prefixes. So Left factoring once again

S → bSS' / a S' → SaA / b A → aS / Sb

This is a left factored grammar.

➢ DIFFERENCES BETWEEN LEFT FACTORING AND LEFT RECURSION

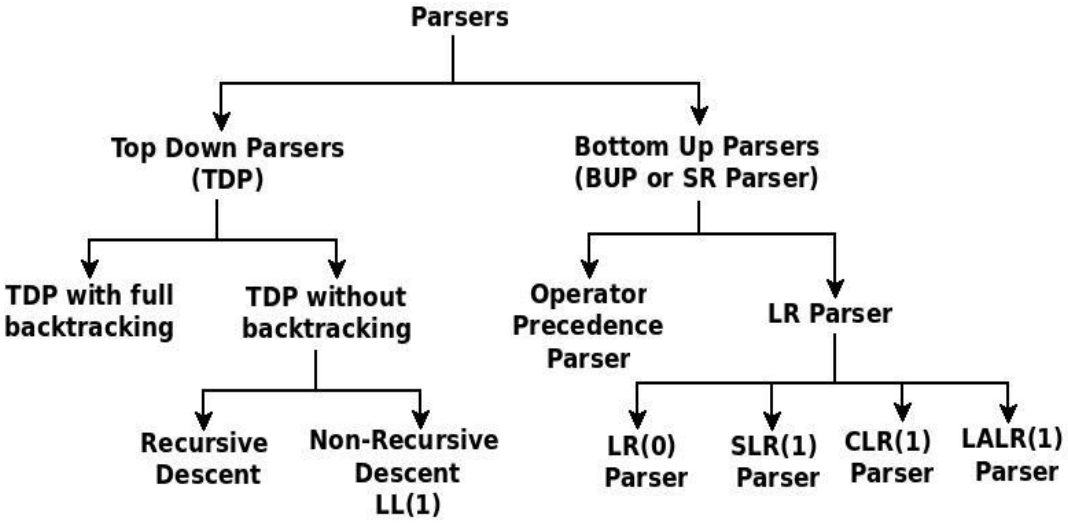
Left Recursion	Left Factoring
A grammar is left recursive if a non-terminal calls itself as the **leftmost symbol** in one of its productions.	A grammar has left factoring when two or more productions of a non-terminal begin with the **same prefix**.
Causes **infinite recursion** in recursive descent parsing.	Creates **choice ambiguity and backtracking**, making it difficult for the parser to decide which production to use.
Purpose of eliminations is to remove infinite recursion.	Its purpose is to postpone the decision until enough input has been read.
`Grammar with Left recursion is A → Aα | β	Grammar with common prefix is A→ aβ1 + aβ2+aβ3+…
Convert left recursion into right recursion.	Extract the common prefix into a new non-terminal.
With Left Recursion E → E + T | T	With common prefix S → aaS | abS| b
After elimination E → T E'E' → + T E' | ε	After left factoring S → aS'|b S' → as|bS

➢ PARSING TYPES

Parsing (also called Syntax Analysis) is the second phase of a compiler. It checks whether the sequence of tokens produced by the lexical analyzer follows the grammatical rules (CFG) of the programming language. If the program is syntactically correct, the parser constructs a parse tree or syntax tree.
Mainly 2 parsing approaches:

1. Top Down Parsing
2. Bottom Up Parsing

In top down parsing, parse tree is constructed from top (root) to the bottom (leaves).
In bottom up parsing, parse tree is constructed from bottom (leaves)) to the top (root).
It can be viewed as an attempt to construct a parse tree for the input starting from the root and creating the nodes of parse tree in preorder.
Pre-order traversal means: 1. Visit the root 2. Traverse left subtree 3. Traverse right subtree.



➢ DIFFERENCE BETWEEN TOP DOWN AND BOTTOM UP PARSING

Top-Down Parsing	Bottom-Up Parsing
Starts from the start symbol and tries to generate the input string.	Starts from the input string and reduces it to the start symbol.
Parse tree is built from Root → Leaves	Parse tree is built from Leaves → Root
Leftmost derivation is used	Rightmost derivation in reverse is used
Basic operation in this is expand non-terminals using productions.	Basic operation is reduce substrings (handles) to non-terminals.
May require backtracking (recursive descent). Predictive parsers avoid it.	No backtracking.
Cannot handle left-recursive grammars. Left recursion must be eliminated.	Can handle left-recursive grammars.
Example : Recursive Descent, Predictive (LL(1)) Parser	Example : Shift-Reduce, Operator Precedence, LR, SLR, LALR, CLR
Implementation is Easier	Implementation is More complex
Usually detects errors earlier while expanding productions.	Usually detects errors during shift/reduce process.
Can struggle with ambiguous grammars.	Better equipped to handle ambiguities through its pattern recognition .
The traversal of parse tree is a pre-order traversal (Root → Left Subtree → Right Subtree)	The traversal of parse tree is a reversal of post order traversal Left Subtree → Right Subtree → Root

Top-down Parser

Example:

E → E+T | T
T → T*F | F
F → id
Input string is id * id

Leftmost derivation →

E ⇒ T
⇒ T * F
⇒ F * F
⇒ id * F
⇒ id * id

Thus this process is like constructing the parse tree from the root (top) towards the leaves.

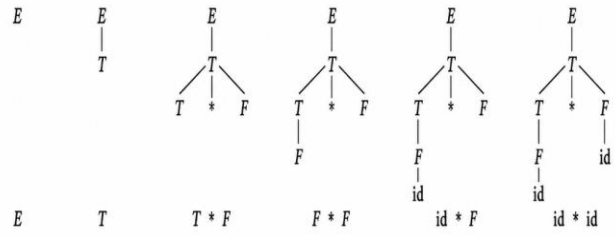


Fig: Top-down parse for id * id

Bottom-Up Parser

Example:

E → E+T | T
T → T*F | F
F → id
Input string is id * id

Rightmost derivation →

E ⇒ T
⇒ T * F
⇒ T * id
⇒ F * id
⇒ id * id

Thus this process is like tracing out the right most derivations in reverse.

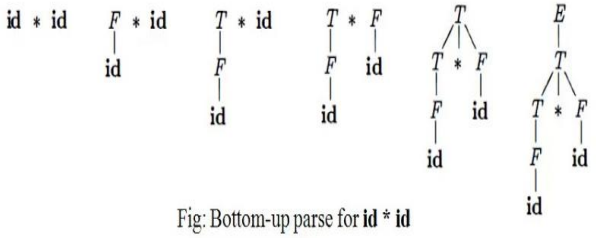


Fig: Bottom-up parse for id * id

➢ TOP-DOWN PARSING

A **Predictive Parser** is a top-down parser that predicts the correct production rule to apply by looking at the next input token (lookahead symbol). It parses the input without backtracking.

➢ RECURSIVE DESCENT PARSING

Recursive descent is a top-down parsing technique that constructs the parse tree from the top and the input is read from left to right.

A Recursive Descent Parser is a top-down parser that consists of a collection of recursive procedures, one for each non-terminal in the grammar. It starts with the start symbol and recursively parses the input to produce a leftmost derivation.

- Start parsing from the start symbol.
- Call the recursive function corresponding to that non-terminal.
- If a production contains another non-terminal, call its corresponding function.
- If a terminal is encountered, match it with the current input token.
- If all tokens are matched successfully, the input is accepted; otherwise, a syntax error is reported.

Advantages

- Simple and easy to understand.
- Suitable for small grammars.
- Easy to implement manually.
- Clearly reflects the grammar structure.

Disadvantages

- May require backtracking, making it slower.
- Cannot parse left-recursive grammars.
- May require left factoring to avoid ambiguity and improve efficiency.
- Less powerful than bottom-up parsers.

Example : Recursive Decent parser for the grammar E→ E+ T | T T → id

After eliminating left recursion

E → T E' E' → + T E' | ε T → id

This grammar is not left recursive, so it is suitable for recursive descent parsing.

```
#include <stdio.h>
#include <stdlib.h>
char lookahead;
void match(char expected_char)
{
   if (lookahead == expected_char)
   {
      lookahead = getchar();
   }
   else
       {
      printf("Error: Expected '%c', but found '%c'\n",  expected_char, lookahead);
      exit(1);
   }
}
E()
{
   T();
   E_dash();
}

E_dash ()
{
   if(lookahead == '+')
   {
      match('+');
      T();
      E_dash ();
   }
   else
   {
            return;           // ε-production
   }
}
T()
{
   if(lookahead == id)
      match(id);
   else
      error();
}
int main()
{
   printf("Enter an arithmetic expression (e.g., i+i*i$): ");
   lookahead = getchar(); // Get the first character

   E(); // Start parsing from the start symbol E

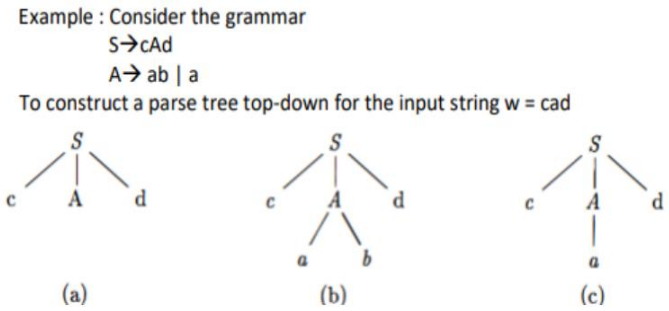
   if (lookahead == '$')  // '$' typically marks the end of input
           {    printf("Parsing Successful!\n");
           }
  else
       {
      printf("Error: Extra characters after parsing\n");
       }
}
```

➢ BACK TRACKING

- **Backtracking** is the process in which a parser returns to a previous point and tries another production rule when the current choice fails to parse the input.
- It is commonly associated with recursive descent parsers that do not use predictive parsing.
- When a grammar has more than one possible production for a non-terminal and the parser cannot determine the correct one using only the next input symbol, it may choose the wrong production. If that happens, it must backtrack and try another alternative.

Example

- Given grammar: S→cAd, A→ab|a
- Input string is **cad**.
- First, parser uses S→cAd.
- It first chooses A→ab, producing **cabd**.
- **cabd** does not match **cad**, so parsing fails.
- Parser goes back to A and tries the alternative A→a.



- Now it produces **cad**, which matches the input. Therefore, the input string is accepted.

➢ FIRST and FOLLOW

FIRST and FOLLOW functions aid in the construction of both top-down and bottom-up parsers for a given grammar G.

FIRST () is a function that specifies the set of terminals that start a string derived from a production rule. It is the first terminal that appears on the right-hand side of the production.

Compute First() Rules:

1. If 'a' is terminal, then FIRST(a) is {a}.
2. If A→ε is a production, then add ε to FIRST(A).
3. For any non-terminal A with production rules A→$\alpha_1|\alpha_2|\alpha_3|\ldots|\alpha_n$, then FIRST(A) = FIRST($\alpha_1$) U FIRST($\alpha_2$) U FIRST($\alpha_3$) U … U FIRST($\alpha_n$)
4. If the production rule is of the form A→$B_1B_2B_3\ldots B_n$, then FIRST(A) = FIRST($B_1B_2B_3\ldots B_n$)
   - If First(B_1) not contains ε then FIRST(A) = FIRST(B_1)
   - If First(B_1) contains ε then FIRST(A) = { First(B_1) – ε } U First(B_2Y_3)
   - If First(Bi) contains ε for all i= 1 to n then add ε to First(A)

FOLLOW()

The FOLLOW set shows the symbols/terminals that can appear right after a non-terminal in a derivation.

Follow() Rules:

1. If A is a starting symbol of given grammar G then FOLLOW(A)={$]
2. For every production B → α A β, where α and β are any string of grammar symbols and A is non-terminal, then everything in FIRST(β) except ε is placed on FOLLOW(A)
3. For every production B → α A, or a production B → α A β, FIRST(β) contains ε, then everything in FOLLOW(B) is added to FOLLOW(A)

Note-**01:**

∈ may appear in the first function of a non-terminal but never appear in the follow function of a non-terminal.

Note-**02:**

Before calculating the first and follow functions, eliminate **Left Recursion** from the grammar, if present.

Note-**03:**

We calculate the follow function of a non-terminal by looking where it is present on the RHS of a production rule.

Example: $A \rightarrow abc \mid def \mid ghi$

Then, we have- First(A) = { a , d , g } Follow(A) = { $}

Example: For a given context-free grammar

S → A B C
A → a | ε
B → b
C → c | ε

FIRST(A) = {a, ε}	FOLLOW(S) = {$}
FIRST(B) = {b}	FOLLOW(A) = {b}
FIRST(C) = {c, ε}	FOLLOW(B) = {c}
	FOLLOW(C) = {$}

Example

S→ABS
A→a
B→b

FIRST(A) = {a}.	FOLLOW(S) = {$}
FIRST(B) = {b}.	FOLLOW(A) = FIRST(B) = {b}
FIRST(S) = FIRST(A) = {a}	FOLLOW(B) = FOLLOW(S) = {$}

➢ LL(1) PARSER

An LL(1) parser is a top-down parser that reads the input from Left to right, constructs the Leftmost derivation, and uses 1 lookahead symbol to decide which production to apply.

The notation LL(1) means:

L → Scan the input from Left to right.

L → Produce the Leftmost derivation.

1 → Use one lookahead token to select the production.

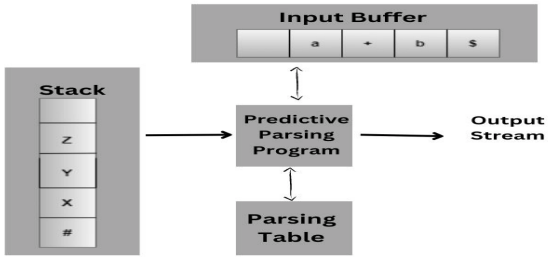
An LL(1) parser is a non-backtracking predictive parser that uses FIRST and FOLLOW sets to construct a parsing table and parse the input using only one lookahead symbol.

An *LL(1) grammar* is a type of context-free grammar that can be parsed using a top-down, predictive parser without backtracking. LL(1) parser, which relies on LL(1) grammars.

Requirements for LL(1) Grammar

- No Left Recursion
- Left Factoring should be performed (if common prefixes exist)
- No ambiguity
- A grammar is LL(1) if and only if its parsing table has a single entry (or no entry, indicating an error) for each cell. Multiple entries would indicate ambiguity or other issues.
- For any two distinct productions A -> α and A -> β, the FIRST sets of α and β should be disjoint (i.e., they should not have any common elements), or one of them must derive the empty string and the other must not derive any string starting with a terminal in FOLLOW(A).
- This property ensures that the parser can uniquely determine which production to apply based on the next input symbol.

Model of LL Parser /Components of LL(1) parser



The parser consists of:

1. Input Buffer – Contains the input string ending with $.
2. Stack – Initially contains $ and the Start symbol ie. $S.
3. Parsing Table – Constructed using FIRST and FOLLOW sets.
4. Parsing Algorithm – Controls parsing using the stack and table.

LL(1) Parsing Table Construction

To construct an LL(1) parsing table for a given grammar:

1. *Remove Left Recursion and Left Factoring*: If the grammar contains left recursion or common prefixes in productions, these must be eliminated or factored out, respectively, to make the grammar suitable for LL(1) parsing.
2. *Compute FIRST and FOLLOW sets*: Calculate the FIRST and FOLLOW sets for each non-terminal in the grammar.
3. Parser table construction

   Step 1: Create a parsing table M.

   Rows represent non-terminals. Columns represent terminals (including $).

   Step 2: For each production A→α:

   1. Compute FIRST(α).
   2. For every terminal a ∈ FIRST(α), where a ≠ ε, place M[A,a]=A→α
   3. If ε ∈ FIRST(α), then for every terminal b ∈ FOLLOW(A), place M[A,b]=A→α
   4. If ε ∈ FIRST(α) and $ ∈ FOLLOW(A), then place M[A,$]=A→α

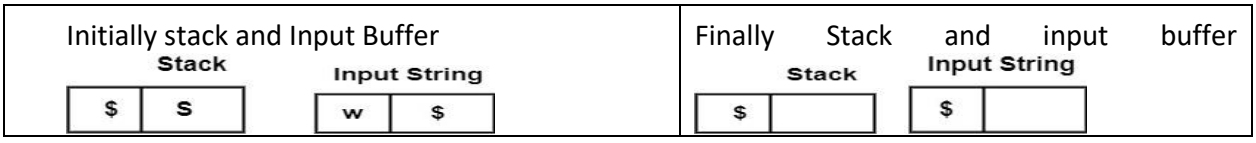
   Step 3: Repeat Steps 1–4 for all productions.

   Step 4: Any remaining empty entries are marked as error.

5. Checking for LL(1) Grammar
   - A simpler way to check for LL(1) is to try to construct the parsing table. If the table can be built without any conflicts, then the grammar is LL(1).
   - If the table has multiple entries in any cell (conflicting productions), then it is not LL(1).

LL(1) Parsing Algorithm

1. Push **$** and the **start symbol** onto the stack.
2. Read the first input symbol (lookahead).
3. Repeat until the stack is empty
   - If Stack top is 'X' and current input symbol is 'a'
   - If the top of the stack is a **terminal**
     - If it matches the lookahead i.e. X== a, pop it and advance the input.
     - Otherwise, report an error.
   - If the top is a **non-terminal**: Lookup for entry M[X,a] in parsing table
     - If M[X,a] contains a production X→ y1y2…yk then
       - Pop X from stack and push y1y2…yk on to stack in reverse order
     - If M[X,a] is empty
       - Report an error and terminate
4. Accept if both the stack and input contain only **$**.



Example : Construct LL(1) parser to parse the string **id+id** using Grammar

E→ E+T | T T → T* F| F

F → (E) | id

Step 1: Eliminate left recurstion from grammar. The resultant grammar is :

E --> TE'

E' → +TE' | ε

T → FT'

T' → *FT' | ε

F → id | €

Step 2: Find first() and follow().		
	First	**Follow**
E --> TE'	{ id, (}	{ $,) }
E' --> +TE' |ε	{ +, ε }	{ $,) }
T --> FT'	{ id, (}	{ +, $,) }
T' --> *FT' | ε	{ *, ε }	{ +, $,) }
F --> id | (E)	{ id, (}	{ *, +, $,) }

Step 3: The parsing table for the above grammar will be:						
	id	**+**	*****	**(**	**)**	**$**
E	E -> TE'			E -> TE'		
E'		E' -> +TE'			E' -> ε	E' -> ε
T	T -> FT'			T -> FT'		
T'		T' -> ε	T' -> *FT'		T' -> ε	T' -> ε
F	F -> id			F -> (E)		

Step 4 : Parsing the string id + id usng parsing table

stack	**input**	**output**
$E	id+id$	E → TE`
$E`T	id+id$	T → FT`
$E`T`F	id+id$	F → id
$ E`T`id	id+id$	
$ E`T`	+id$	T` → ε
$ E`	+id$	E` → +TE`
$ E`T+	+id$	
$ E`T	id$	T → FT`
$ E`T`F	id$	F → id
$ E`T`id	id$	
$ E`T`	$	T` → ε
$ E`	$	E` → ε
$	$	accept

Example :

Check the given grammar is LL(1) or not

Example 2

S → iEtSS' | a S' → es | ε E → b

	First	**Follow**
S → iEtSS' | a	{i,a}	{$, e}
S' → es | ε	{e, ε}	{$, e}
E → b	[b}	{t}

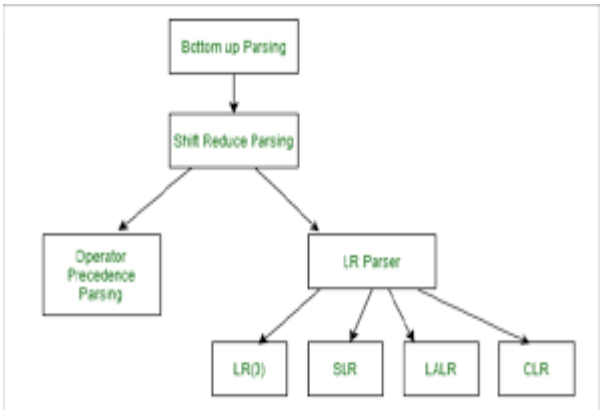
Parsing table

	a	b	e	t	i	$
S	s→a				s->iEtss'	
s'			S'→ es S'→ ε			S'→ ε
E		E → b				

The cell [s', e] has two entries which results ambiguity so the given grammar is not LL(1)

BOTTOM-UP PARSING

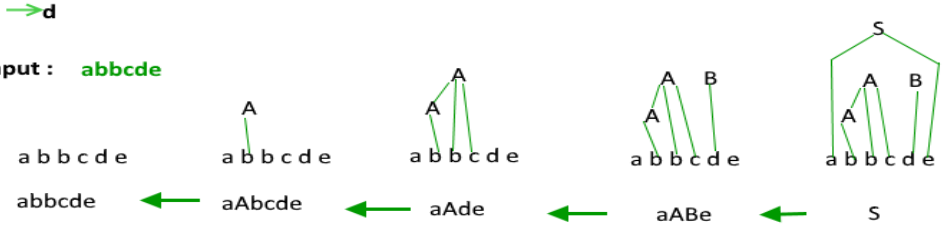
Bottom-up parsing in compiler design starts from the input tokens and works towards the root of the parse tree, effectively reducing the input to the starting symbol of the grammar. This approach builds the parse tree from the bottom (leaf nodes) upwards, using production rules in reverse to find the original structure. It's also known as shift-reduce parsing because it utilizes these two primary actions to analyze the input. i.e. SHIFT and REDUCE



Example

S → aABe
A → Abc/b
B → d

Input : abbcde



a) Shift Operation

A **shift** operation moves the **next input symbol** from the input string onto the parser's **stack**. Its purpose is

- Read the next input symbol.
- Build a handle that can later be reduced.

b) Reduce Operation

- In bottom-up parsing, a reduce operation replaces a sequence of symbols (called a handle) on the top of the stack by the corresponding non-terminal using a grammar production.
- At each reduction step, a specific substring matching the body of a production is replaced by the non-terminal at the head of the production.
- In derivation, a non-terminal in a sentential form is replaced by the body of one of its productions. A reduction is reverse of a step in derivation.

Example

Grammar is

S→aABe
$A \rightarrow A\ b\ c$
$A \rightarrow b$
$B \rightarrow d$

and the input string abbcde

The Reduction:

Prod'n.	Sentential Form
3	a b bcde
2	a Abc de
4	aA d e
1	aABe
–	S

Rightmost Derivation:

$S \Rightarrow aABe$
$\Rightarrow aAde$
$\Rightarrow aAbcde$
$\Rightarrow abbcde$

Notice that the reduction is actually reverse of the rightmost derivation.

➢ HANDLE PRUNING

Handle pruning is the process used by a bottom-up parser to construct the parse tree by repeatedly finding a handle in the input string and reducing it to its corresponding non-terminal until only the start symbol remains**.**

It essentially reverses the process of deriving a string from a grammar.

Removing the children of the left-hand side non-terminal from the parse tree is called **Handle Pruning**.

A rightmost derivation in reverse can be obtained by handle pruning.

A **handle** is a substring of a right sentential form that matches the right-hand side (RHS) of a production and whose reduction represents one step in the reverse of a rightmost derivation**.**

Example

- Grammar rules define E → E + T | T T → T * F | F F → (E) | id

  string: W = id * id

Step	Current String	Handle	Reduction
1	**id** * id	id	F → id
2	**F** * id	F	F → T
3	T * **id**	id	F → id
4	**T * F**	T * F	E → T * F
5	E	Start symbol obtained	Accept

➢ SHIFT-REDUCE PARSING

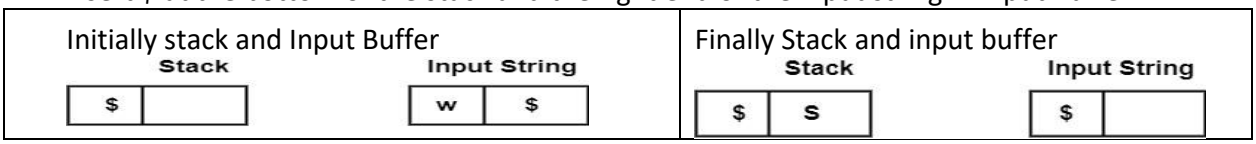
Shift-reduce parsing is a bottom-up parsing technique used in compiler design to analyze the syntax of a programming language. It uses a stack to store grammar symbols and an input buffer to hold the string being parsed.

Two data structures are required to implement a shift-reduce parser-

- A **Stack** is required to hold the grammar symbols.
- An **Input buffer** is required to hold the string to be parsed.

There are the various steps of Shift Reduce Parsing which are as follows –

- It uses a Stack and an Input Buffer.
- Insert $ at the bottom of the stack and the right end of the input string in Input Buffer.



Algorithm for Shift-Reduce Parser

1. Initialize stack with $.
2. Append $ to the input string.
3. Repeat the following steps until parsing is complete:
   a) Shift the next input symbol onto the stack.
   b) If the top of the stack matches the RHS of a production, reduce it to the corresponding LHS.
   c) If the stack contains $S and the input is $, accept the string.
   d) Otherwise, if no valid action exists, report a syntax error.

Example of the shift-reduce parser. Consider that we have a string **id * id** and the grammar for the input string is:

E ->E + T | T T -> T * F | F F -> (E) | id

In the shift-reduce parsing a handle always appears on the top of the stack. The handle is a substring that matches the body of production in the grammar. The handle must never appear inside the stack.

STACK	INPUT	ACTION
$	$\mathbf{id}_1 * \mathbf{id}_2 \$$	shift
$\$ \mathbf{id}_1$	$* \mathbf{id}_2 \$$	reduce by $F \rightarrow \mathbf{id}$
$\$ F$	$* \mathbf{id}_2 \$$	reduce by $T \rightarrow F$
$\$ T$	$* \mathbf{id}_2 \$$	shift
$\$ T *$	$\mathbf{id}_2 \$$	shift
$\$ T * \mathbf{id}_2$	$\$$	reduce by $F \rightarrow \mathbf{id}$
$\$ T * F$	$\$$	reduce by $T \rightarrow T * F$
$\$ T$	$\$$	reduce by $E \rightarrow T$
$\$ E$	$\$$	accept

Example : Input String **id * id + id** for using same grammar of previous example

Stack	Input	Action
$	id*id+id $	Shift id
$ id	*id+id $	Reduce E -> id
$ E	*id+id $	Shift *
$ E *	id+id $	Shift id
$ E * id	+id $	Reduce E -> id
$ E * E	+id $	Reduce E-> E * E
$ E	+id $	Shift +
$ E +	id $	Shift id
$ E + id	$	Reduce E -> id
$ E + E	$	Reduce E -> E + E
$ E	$	Accept

Disadvantages of Shift-Reduce Parser:

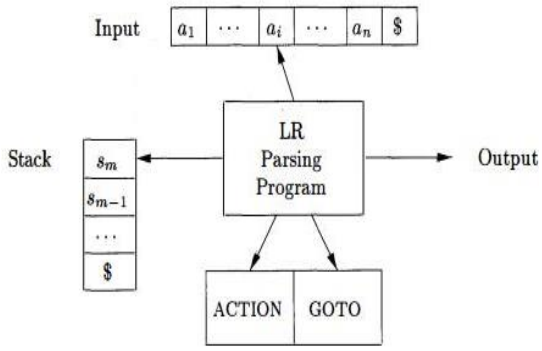
- It may encounter shift-reduce conflicts, where the parser cannot decide whether to shift or reduce.
- It may encounter reduce-reduce conflicts, where multiple reductions are possible.
- It cannot parse all context-free grammars.
- It has difficulty handling ambiguous grammars.
- Error detection and recovery are limited compared to advanced LR parsers.

➢ LR PARSER

An LR parser is a bottom-up parser that scans the input from left to right and constructs the rightmost derivation in reverse by repeatedly performing shift and reduce operations.

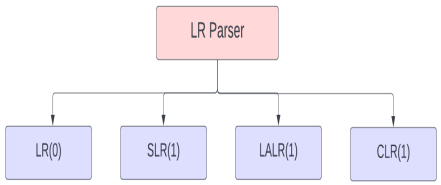
Components of an LR Parser

An LR parser consists of:

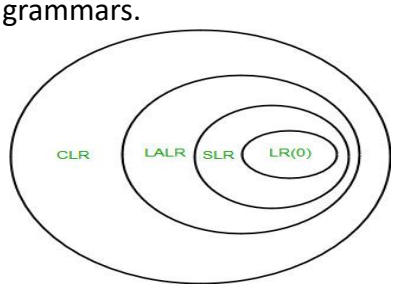


1. Input Buffer : Stores the input tokens.
2. Stack : Stores grammar symbols and parser states.
3. Parsing Table :
   Contains ACTION Table (Shift, Reduce, Accept, Error), GOTO Table (State transitions for non-terminals)
4. Parser Driver : Controls the parsing process using the stack and parsing table.

Types of LR Parsers



1. LR(0) Parser
   - Meaning: Uses 0 lookahead symbols.
   - Mechanism: Makes parsing decisions based entirely on the current state and the current symbol on the stack.
   - Pros/Cons: Very fast but highly restrictive. It is prone to shift-reduce and reduce-reduce conflicts on complex grammars.
2. SLR (Simple LR) Parser
   - Meaning: The simplest form of lookahead parser.
   - Mechanism: Uses 1 lookahead symbol and the FOLLOW sets of the non-terminals to resolve conflicts during reduction.
   - Pros/Cons: Easy to implement and more powerful than LR(0), but it can still fail to resolve conflicts in highly complex grammars.



3. CLR (Canonical LR) Parser
    - Meaning: The most powerful standard LR parser.
    - Mechanism: Uses 1 lookahead symbol associated directly with the production items to make highly precise parsing decisions.
    - Pros/Cons: Can parse almost any deterministic context-free grammar. However, it produces a massive parsing table that requires significant memory.
4. LALR (Look-Ahead LR) Parser
    - Meaning: The industry standard for most parser generators (such as GNU Bison or YACC).
    - Mechanism: Optimizes the CLR parser by merging states that have the same core items but different lookahead symbols.
    - Pros/Cons: Retains almost all the parsing power of CLR but with a table size comparable to SLR

➢ LR(0) PARSER

In LR(0) ,

L → Scan the input from Left to Right.

R → Construct the Rightmost derivation in reverse.

The "(0)" signifies that the parser makes its decisions (shift or reduce) based solely on the current state and the next input symbol, without using any lookahead symbols.

To perform LR(0) parsing, take grammar as input and do the following:

LR(0) items:

An LR(0) item is a production with a dot (•) indicating the parser's current position. For example, production A → XYZ yields the four items:

A → .XYZ A → X.YZ A → XY.Z A → XYZ.

Closure Operation:

- The closure of an LR(0) item is the process of adding all the productions that may need to be parsed next.
- Start with the given LR(0) item(s).
- If an item has a dot (·) before a non-terminal, add all productions of that non-terminal with the dot at the beginning.
- Repeat this until no new items can be added.

Goto Operation:

- The Goto(I, X) function moves the dot over the grammar symbol X in every applicable item of state I, and then computes the closure of the resulting items.
- It tells us which state the parser moves to after recognizing the symbol X.

Canonical Collection of LR(0) items

- The canonical collection of LR(0) items is the complete set of all LR(0) item sets (states) generated by repeatedly applying the Goto function, starting from the initial closure.
- Each state in the DFA is one LR(0) item set.

Steps to Construct an LR(0) Parser:

1. Augment the grammar by adding a new start production (S' → S) and assign a number to each production
2. Construct the LR(0) items by placing a dot (•) at every possible position in each production.
3. Compute the closure of the initial item to obtain the initial state.
4. Construct the canonical collection of LR(0) items using the GOTO function.
5. Construct parsing table with 2 sub tables i.e. Action and GoTo .

- Construct the ACTION table by entering Shift, Reduce, Accept, or Error actions.
- Construct the GOTO table for transitions on non-terminals.

6. Use the ACTION and GOTO tables to parse the input string through shift and reduce operations.

Parsing Table for LR(0) Parser

- The parsing table is a two-dimensional array consisting of two tables- the ACTION table and the GOTO table. Create a table with rows representing states and columns representing terminals and non-terminals.
- The "action" part of the table handles terminals and the "goto" part handles non-terminals.
- For each state, perform the following to fill LR(0) parsing table using these four rules:
  - i. Dot before a terminal ⟶ SHIFT
    Fill Action[State **j** , terminal] = S j i.e. Shift to State j
  - ii. Dot before a non-terminal ⟶ GOTO
    GoTo[State **j** , Non-termina] = j where **j** is state number
  - iii. Dot at the end of a production ⟶ REDUCE on **all** terminal columns
    Fill Action[State **j** , terminal] = R**i** where **i** is production number
    Reduce with i^{th} production
  - iv. Dot before $ i.e. (S' → S.$) ⟶ ACCEPT
    Fill Action[State **j** , $] = Accept where **j** is state number
- Blank entries represent error states.
- If there are **multiple** entries in a single cell (e.g., shift-reduce or reduce-reduce conflicts), the grammar is not LR(0).

LR-Parsing Algorithm

The LR (Left-to-Right) algorithm, commonly referred to in the context of parsing, is a technique used to analyze a sequence of input symbols to determine its grammatical structure with respect to a given formal grammar.

1. Initialization

   Push the initial state 0 onto the stack.

   Set the input pointer to point to the first character of the input string.

2 Repeat the following steps until the parser explicitly finishes:

   Let 'S' be the state currently on top of the stack. Let 'a' be the current symbol pointed to in the input buffer. Look up the table entry ACTION[S, a]

   i. If ACTION[S, a] = Sj
      1. Push the input symbol 'a' onto the stack. Push the new state 'j' onto the stack.
      2. Advance the input pointer to the next symbol

   ii. If ACTION[S, a] = Ri " Let i^{th} production is A → β":
      1. Let k be the length of the string β (the right-hand side of the production rule).
      2. Pop (2 × k) items from the stack
      3. Let S' be the state that is now on top of the stack after the pop operation.
      4. Push the non-terminal symbol A onto the stack.
      5. Look up GOTO[S', A] to get the next state and push that next state onto the stack.
      6. Note: Do not advance the input pointer during a reduction

   iii. If ACTION[S, a] ="Accept":

      Stop processing. The input string is syntactically valid and successfully parsed.

   iv. If ACTION[S, a] = Empty

      Stop processing. The input string contains a syntax error and is rejected

Example of LR(0) Parsing:

Perform LR(0) parsing for the following grammar: S → AA A → aA | b

Step 1: Convert given grammar into augmented grammar.

Augmented grammar:

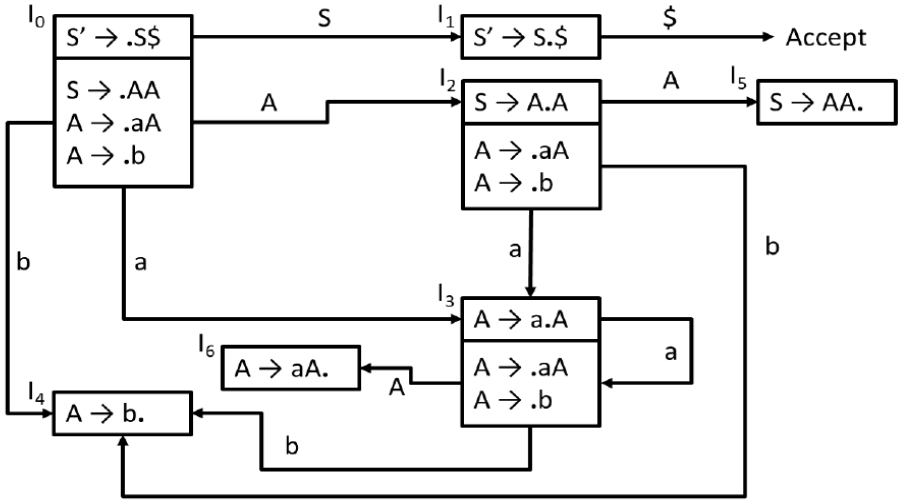
0 S' →S$ — — — — (Adding the Additional Production)

1 S → AA

2 A → aA

3 A → b

Step 2: Find LR (0) items.



Step 3: LR(0) parsing table

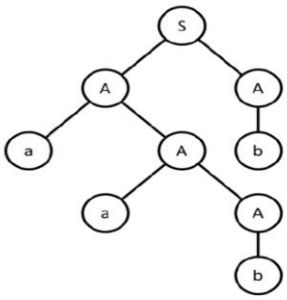
State	Action			Goto	
	a	**b**	**$**	**S**	**A**
I0	S3	S4		1	2
I1			Accept		
I2	S3	S4			5
I3	S3	S4			6
I4	R3	R3	R3		
I5	R1	R1	R1		
I6	R2	R2	R2		

Note: Blank entries represent error states.

Step 4: Stack implementation. Check whether the input "**aabb**" is valid or not.

Stack	Input	Action
$0	a a b b $	Shift 3
$0 a3	a b b $	Shift 3
$0 a3 a3	b b $	Shift 4
$0 a3 a3 b4	b $	Reduce A → b
$0 a3 a3 A6	b $	Reduce A → aA
$0 a3 A6	b $	Reduce A → aA
$0 A2	b $	Shift 4
$0 A2 b4	$	Reduce A → b
$0 A2 A5	$	Reduce S → AA
$0 S1	$	Accept

Step 5: Parse Tree



Parse Tree for the input: aabb

➢ CONFLICTS IN LR PARSERS

A conflict in an LR parser occurs when more than one parsing action is possible for the same state and input symbol while constructing the LR parsing table.

There are two types of conflicts:

1. Shift-Reduce (SR) Conflict

An SR conflict occurs when the parser has to choose between a shift action and a reduce action for the same state and input symbol.

This is a problem in which the Parser cannot decide whether to SHIFT the next input symbol or REDUCE the symbols on the stack.

Both SHIFT and REDUCE entries occur in the same ACTION table cell.

Commonly occurs due to ambiguous grammars.

Example:

Grammar is E→E+E | id and string `id + id+ id`

Suppose the stack contains: E+E and the next input symbol is +.

The parser may: **Reduce:** E+E→E or **Shift:** Push the next + onto the stack.

So, an SR conflict occurs when both SHIFT and REDUCE actions are possible for the same state and input symbol.

2. Reduce-Reduce (RR) Conflict

An RR conflict occurs when the parser has to choose between two or more reduce actions for the same state and input symbol.

This is a problem in which Parser cannot decide which production to use for reduction.

Two or more REDUCE actions occur in the same ACTION table cell.

Usually indicates ambiguity or insufficient information in the grammar.

Example:

Grammar is S→A|B A→a B→a

After reading symbol 'a', the parser may reduce using: A→a or B→a

So, an RR conflict occurs when two or more REDUCE actions are possible for the same state and input symbol.

➢ SLR(1) PARSER

An SLR(1) parser (Simple LR parser) is a bottom-up parser that reads the input from left to right and constructs the rightmost derivation in reverse. It uses one lookahead symbol and the FOLLOW sets of non-terminals to decide reduce actions. SLR(1) means

S → Simple
L → Left-to-right scanning of the input
R → Rightmost derivation in reverse
1 → One lookahead symbol

An LR(0) parser performs a reduce action for all terminals whenever it encounters a completed item (A → α •). This can cause shift–reduce or reduce–reduce conflicts.

An SLR(1) parser reduces only when the current input symbol belongs to the FOLLOW set of the left-hand-side non-terminal, thereby resolving many of these conflicts.

Drawbacks of SLR(1)

- The SLR Parser discussed in the earlier class has certain flaws.
- On single input, State may be included a Final Item and a Non- Final Item. This may result in a Shift-Reduce Conflict .
- A State may be included Two Different Final Items. This might result in a Reduce-Reduce Conflict
- SLR(1) Parser reduces only when the next token is in Follow of the left-hand side of the production.
- SLR(1) can reduce shift-reduce conflicts but not reduce-reduce conflicts
- These two conflicts are reduced by CLR(1) Parser by keeping track of lookahead information in the states of the parser.

Steps to Construct an SLR(1) Parser

1. Augment the grammar.
2. Construct the canonical collection of LR(0) items.
3. Compute the FIRST and FOLLOW sets.
4. Construct the ACTION table.
   - Shift: For transitions on terminals.
   - Reduce: For completed items (A → α •), enter reductions only in the columns corresponding to FOLLOW(A).
   - Accept: For S' → S • under $.
5. Construct the GOTO table for non-terminal transitions.
6. Use the ACTION and GOTO tables to parse the input.

Parsing Table for SLR(1) Parser

- The parsing table is a two-dimensional array consisting of two tables- the ACTION table and the GOTO table. Create a table with rows representing states and columns representing terminals and non-terminals.
- The "action" part of the table handles terminals and the "goto" part handles non-terminals.
- For each state, perform the following to fill LR(0) parsing table using these four rules:
  - i. Dot before a terminal ⟶ SHIFT
    Fill Action[State **j** , terminal] = S j i.e. Shift to State j
  - ii. Dot before a non-terminal ⟶ GOTO
    GoTo[State **j** , Non-termina] = j where **j** is state number
  - iii. Dot at the end of a production ⟶ REDUCE only in the columns corresponding to **FOLLOW**(A).
    Fill Action[State **j** , terminal] = R**i** where **i** is production number
    Reduce with i^{th} production
  - iv. Dot before $ i.e. (S' → S.$) ⟶ ACCEPT
    Fill Action[State **j** , $] = Accept where **j** is state number
- Blank entries represent error states.
- If there are **multiple** entries in a single cell (e.g., shift-reduce or reduce-reduce conflicts), the grammar is not LR(0).

Example of SLR(1) Parsing:

Perform **SLR**(0) parsing for the following grammar:

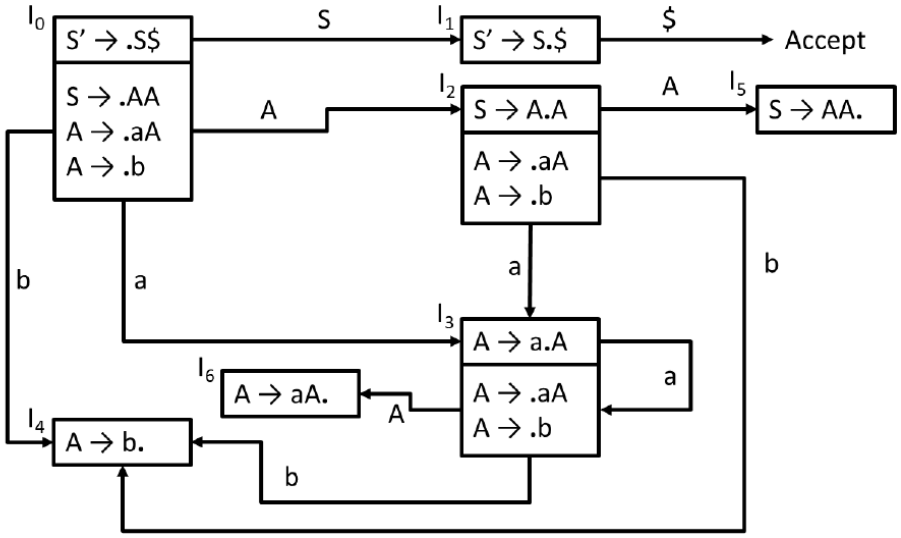
S → AA A → aA | b

Step 1: Convert given grammar into augmented grammar.

Augmented grammar:

S' →S$ S → AA A → aA A → b

Step 2: Find LR (0) items.



STEP3: Find FOLLOW of LHS of production

FOLLOW(S) = { $ }

FOLLOW(A) = { a,b,$ }

Step 3: SLR(1) Parsing table

State	Action			Goto	
	a	**b**	**$**	**S**	**A**
I0	S3	S4		1	2
I1			Accept		
I2	S3	S4			5
I3	S3	S4			6
I4	R3	R3	R3		
I5			R1		
I6	R2	R2	R2		

LR(0) parsing table (for comparison)

State	Action			Goto	
	a	**b**	**$**	**S**	**A**
I0	S3	S4		1	2
I1			Accept		
I2	S3	S4			5
I3	S3	S4			6
I4	R3	R3	R3		
I5	R1	R1	R1		
I6	R2	R2	R2		

➢ CLR(1) PARSER

CLR(1) (Canonical LR(1)) parser is a bottom-up parser that uses LR(1) items (items with one lookahead symbol) to construct parsing tables. It is the most powerful parser among the LR family because it can parse almost all deterministic context-free grammars without conflicts. In CLR(1)

C → Canonical (uses the complete set of LR(1) items)

L → Scan the input from Left to Right R → Construct the Rightmost derivation in reverse

1 → Use 1 lookahead symbol

- LR(1) item = LR(0) item + look ahead

- The look ahead is used to determine that where we place the final item. The look ahead always add $ symbol for argument production
- CLR parsing use the canonical collection of LR(1) items to build the CLR(1) parsing table
- CLR(1) parsing table produces the more number of states as compared to the SLR(1) parsing
- In the CLR(1), we place the reduce node only in the look ahead symbols

Steps to Construct an CLR(1) Parser

1. Augment the given grammar by adding a new start production.
2. Construct the initial LR(1) item Start with the item S'
3. Compute the Closure of LR(1) items using:
   - LR(1) closure rules and Lookahead symbols using FIRST( ).
4. Compute GOTO transitions for all grammar symbols.
5. Repeat Closure and GOTO operations until no new LR(1) states are generated.
6. Construct the CLR(1) parsing table:
   - Create ACTION table for terminals and Create GOTO table for non-terminals.
7. Fill the parsing table entries:
   - Shift (S) Reduce (R) Accept (acc) Error
8. Check for parsing conflicts:
   - Shift-Reduce conflict and Reduce-Reduce conflict
9. Use the CLR(1) parsing table to parse the input string.

LR(1) item

An LR(1) item is an LR(0) item with one lookahead symbol. The lookahead tells the parser under which input symbol a reduction is allowed.

To Find Closures

In LR(0) and SLR(1), items we have used are called LR(0) items which are of the form (A → .BCD or A → B.CD etc.).

In CLR(1) and LALR(1), items which we will use are called as LR(1) items. These are of the form (A → .BCD, α). Here α is a Lookahead symbol which represents the terminals after A .

To find Lookahead Symbol

1. We start with item (S'→ .S). It has Lookahead symbol $ as we assume that string ends at symbol $. So complete LR(1) item becomes (S'→ .S, $).

2. If (A->.BCD, α) is item in closure, then next item will be (B->.DEF, FIRST(CDα).

Similarly,

If (A->B.CD, α) is item in closure, then next item will be (C->.GHI, FIRST(Dα).

If (A->BC.D, α) is item in closure, then next item will be (C->.JKL, FIRST(α).

3. When we have to find first item of next closure, then Lookahead symbols are retained from the previous closure.

Example: CLR(1) Parser

Perform LR(0) parsing for the following grammar:

S → AA A → aA | b

Step 1: Convert given grammar into augmented grammar.

Augmented grammar:

S' →S$

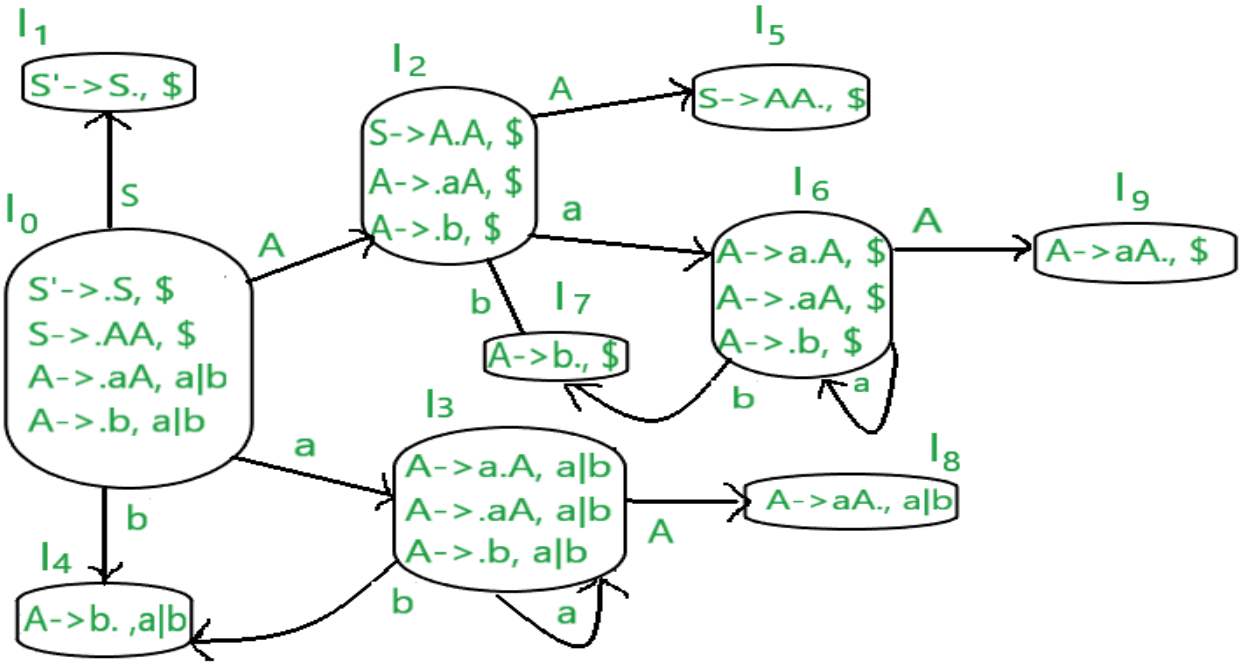
S → AA

A → aA

A → b

Non-Terminal	FIRST	FOLLOW
S	{ a, b }	{ $ }
A	{ a, b }	{ a, b, $ }

STEP 2 - Find LR(1) collection of items



CLR(1) Parsing Table

State	Action			Goto	
	a	**b**	**$**	**S**	**A**
I0	S3	S4		2	1
I1			ACCEPT		
I2	S6	S7		5	
I3	S3	S4		8	
I4	R3	R3			
I5			R1		
I6	S6	S7		9	
I7			R3		
I8	R2	R2			
I9			R2		

➢

➢ LALR(1) PARSER

A LALR(1) parser (Look-Ahead LR parser) is a bottom-up parser that combines the advantages of SLR(1) and CLR(1). It uses one lookahead symbol like CLR(1), but merges compatible CLR(1) states to reduce the size of the parsing table.

It is the most commonly used LR parser in practice because it provides good parsing power while keeping the parsing tables relatively small. In LALR(1)

- **L** → Scan the input from **Left to Right**
- **A** → **Look-Ahead**
- **LR** → Construct the **Rightmost derivation in reverse**
- **1** → Use **1 lookahead symbol**

The CLR(1) parser is very powerful but generates many states, leading to: Large parsing tables, High memory usage, More construction time

The LALR(1) parser solves this by merging CLR(1) states with identical LR(0) cores, resulting in: Fewer states, Smaller parsing tables

Steps to Construct LALR(1) Parser

1. Augment the given grammar by adding a new start production.
2. Construct the CLR(1) item sets using:
   - Closure operation
   - GOTO operation
3. Identify CLR(1) states having the same LR(0) core (same items ignoring lookahead symbols).
4. Merge the CLR(1) states with the same LR(0) core by combining their lookahead symbols.
5. Construct the LALR(1) parsing table:
   - Fill the ACTION table using terminal symbols and GOTO table using non-terminal symbols.
6. Check for conflicts:
   - Shift-Reduce conflict
   - Reduce-Reduce conflict
7. Use the LALR(1) parsing table to parse the input string using:
   - Shift
   - Reduce
   - Accept
   - Error actions

Example: LALR(1) Parser

Perform LALR(0) parsing for the following grammar:

S → AA A → aA | b

Step 1: Convert given grammar into augmented grammar.

Augmented grammar:

S' →S$ — — — — (Adding the Additional Production)

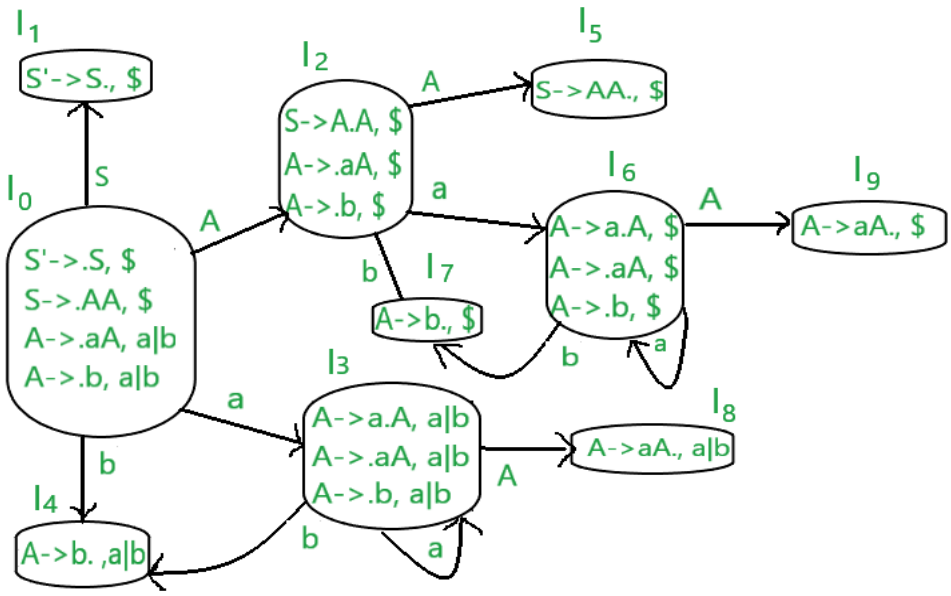
S → AA

A → aA

A → b

Non-Terminal	FIRST	FOLLOW
S	{ a, b }	{ $ }
A	{ a, b }	{ a, b, $ }

STEP 2 - Find LR(1) collection of items



CLR(1) Parsing Table

State	Action			Goto	
	a	**b**	**$**	**S**	**A**
I0	S3	S4		2	1
I1			ACCEPT		
I2	S6	S7		5	
I3	S3	S4		8	
I4	R3	R3			
I5			R1		
I6	S6	S7		9	
I7			R3		
I8	R2	R2			
I9			R2		

You will find there some closures/states which are same in all aspects except Lookahead symbol. Due to this, CLR(1) has many Closures/States which in turn increases the processing time.
In LALR(1), we merge such closures/states. For e.g. in above diagram, we can merge (I3 and I6), (I4 and I7) and (I8 and I9). So we get new closures/states I36, I47 and I89 respectively

Closures	Action			Goto	
	a	**b**	**$**	**A**	**S**
0	s36	s47		2	1
1			Accept		
2	s36	s47		5	
36	s36	s47		89	
47	r3	r3	r3		
5			r1		
89	r2	r2	r2		

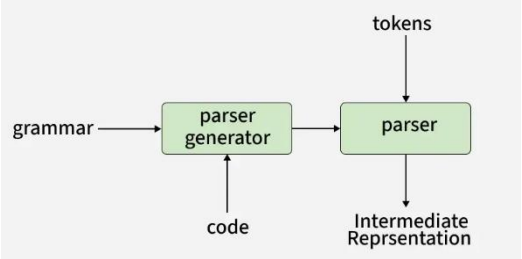
Finally, LALR(1) Parsing Table

State	Action			Goto	
	a	**b**	**$**	**S**	**A**
I0	S3	S4		2	1
I1			ACCEPT		
I2	S3	S4		5	
I3	S3	S4		8	
I4	R3	R3	R3		
I5			R1		
I6	R2	R2	R2		

.

➢ PARSER GENERATORS

A parser generator is a software tool that automates the creation of a parser.



A parser generator is a software tool that automatically creates a parser from a given context-free grammar (CFG) and produces the code for syntax analysis.

Instead of manually writing a parser, the programmer provides:

Grammar rules of the programming language.

Token information from the lexical analyzer.

The parser generator automatically generates a parser program.

There is enough interaction between the lexical phase and the syntactic phase, where the lexer issues a token based on a request from the parser.

The following are some of the tools available for parsing:

ANTLR (ANother Tool for Language Recognition) tool generates LL(k) parsers

YACC (Yet Another Compiler Compiler) generates LALR(1) parsers

Bison – It is an improved version of YACC

A parser generator is a program whose inputs are:

1. Context Free Grammar 2. Token information 3.Semantic Actions

Outputs of parser generator are

1. Parser code 2. Parsing Table

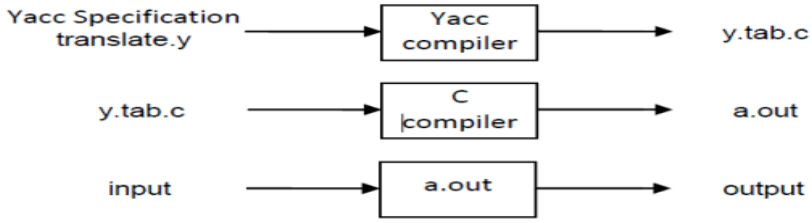
➢ THE PARSER GENERATOR YACC

YACC (Yet Another Compiler-Compiler) is a powerful tool used to generate parsers for compilers. It was designed to produce a parser from a given grammar specification, which is a set of rules defining the syntax of a particular language.

The parser generated by YACC is an LALR parser, operating from left to right and trying to derive the rightmost element of a sentence in a sentence structure according to the grammar.

YACC works in *3* main parts: declarations, translation rules, and supporting C routines.

declarations
%%
translation rules
%%
supporting C routine



i) Declarations

Contains: Header files, Token declarations, Variables

Header files and global variable declarations are enclosed %{…….%}

Tokens are declared with %token

Example %token NUMBER %token ID

ii) Translation Rules

The translation rules section contains grammar definitions i.e. Contains CFG rules with semantic actions.

These rules define how the parser should interpret sequences of tokens. Each rule in YACC has a string specification that resembles a production of a grammar.

```
%%
/* rules */
....
%%
```

Example :

```
expr : expr '+' expr
    {
     $$ = $1 + $3;
    }
    | NUMBER
    ;
```

expr	→	non-terminal
+ and NUMBER	→	terminals
{ }	→	semantic actions

iii) Supporting C Routines

It Contains: Main function, Error handling routines, User-defined functions

If the yylex() function is not defined in the auxiliary routines sections, then it should be included with #include "lex.yy.c". If the YACC file contains the main() definition, it must be compiled to be executable.

Example :

```
int main()
{
  yyparse();
}
```

Steps to Write and Execute a YACC Program

1. Write the YACC program with extension **.y** containing:
   - Declaration section
   - Grammar rules section
   - Supporting C code section
2. Write the Lex specification file with extension **.l** to generate tokens.
3. Generate the lexical analyzer using Lex:

   ```
   lex  filename.l
   ```

   - It generates *lex.yy.c*
4. Generate the YACC parser using YACC:

   ```
   yacc -d  filename.y
   ```

   - It generates:
     - y.tab.c
     - y.tab.h
5. Compile the generated Lex and YACC files:

   ```
   gcc  y.tab.c  lex.yy.c
   ```

   It generates

   ```
   a.out
   ```
6. Execute the generated parser:

   ```
   ./a.out
   ```
7. Provide the input string and observe the parser output.

Yacc Program to recognize a valid arithmetic expression that uses +, -, * and /

```
// arith.l
%{
#include<stdio.h>
#include "y.tab.h"
%}

%%
[a-zA-Z]+ return VARIABLE;
[0-9]+ return NUMBER;
[\t] ;
[\n] return 0;
. return yytext[0];
%%

int yywrap()
{
return 1;
}
```

How To Run:

```
$ yacc -d arith.y
$ lex arith.l
$ gcc lex.yy.c y.tab.c
$ ./a.out
```

```
// arith.y
%{
   #include<stdio.h>
%}
%token NUMBER
%token VARIABLE

%left '+' '-'
%left '*' '/' '%'
%left '(' ')'

%%

S: VARIABLE'='E {
      printf("\nEntered arithmetic expression is Valid\n\n");
      return 0;
    }
E:E'+'E
 |E'-'E
 |E'*'E
 |E'/'E
 |E'%'E
 |'('E')'
 | NUMBER
 | VARIABLE
;

%%

void main()
{
  printf("\nEnter Any Arithmetic Expression which can have
operations +, -,*,/,%  and ( ) \n");
  yyparse();
}

void yyerror()
{
  printf("\nEntered arithmetic expression is Invalid\n\n");

}
```